

KI.AI — AI-Powered Yoga & Posture Intelligence

Complete Production Architecture & Technical Report

Executive Overview:

This document constitutes the comprehensive technical and operational report for **KI.AI**. It details the markerless 3D computer vision pipeline, 2D/3D kinematic vector calculations, real-time state machines, prioritized speech queue engine, 38 REST API routes, 12 SQLite tables, commercial Razorpay payment verification, and 81 automated tests.

Target Repository: `/Users/ashik/Downloads/yoga-main`
Software Version: 1.0.0 (Production Release)
Verification Status: 81/81 Pytest Automated Tests Passing (100%)



Author: Lead Product Engineer & Technical Architecture Specialist

Technology Stack: Python 3.9+, MediaPipe Pose, OpenCV, Flask, SQLite3, PySide6, Razorpay SDK

Date: October 2026 • Antigravity Production Release

2

Table of Contents

Section 1	Project Overview, Mission & Core Problems Solved
Section 2	Complete Repository Directory Structure & File Responsibility
Section 3	Frontend Architecture, Warm Editorial Design & View Catalog
Section 4	Backend Architecture & Flask REST API Guide (38 Endpoints)
Section 5	Pose Detection, Vector Angle Calculation & Scoring Pipeline
Section 6	AI Voice Coach, Priority Queue & Conversational System
Section 7	Session Planner Engine & Guided Learning System
Section 8	Database Structure, Relational Schema & Indexes (12 Tables)
Section 9	Supported Asana Catalog, Visual Gallery & Surya Namaskar Flow
Section 10	Authentication, User Profiles & Commercial SaaS Payments
Section 11	Progress Telemetry, Joint Heatmaps & Analytics
Section 12	Automated Testing Inventory (81 Tests), Status Matrix & Limits
Section 13	End-to-End System Data Flow & Sequence Diagrams
Section 14	Master Function and Class Inventory

SECTION 1 — PROJECT OVERVIEW

1.1 Mission & Problem Statement

KI.AI is an intelligent, dual-environment (Web & Desktop) computer-vision platform designed to act as a real-time, interactive **Personal AI Yoga Teacher**. Traditional digital yoga instruction relies on static, pre-recorded video streaming where students receive no corrective feedback, leading to common movement compensation errors, joint hyperextension, reduced workout efficacy, and increased risk of musculoskeletal injury.

KI.AI solves these problems by providing:

1. **Real-Time Markerless Motion Capture:** Extracts 33 3D body landmarks via MediaPipe Pose directly from consumer webcams or smartphone cameras without specialized sensors or wearables.
2. **Deterministic Biomechanical Kinematics:** Computes exact joint angles (knees, hips, shoulders, elbows, spine, ankles) and compares them against codified yoga anatomical guidelines.
3. **Multi-Stage Posture Evaluation & Scoring:** Dynamically distinguishes between active pose matching, anatomical misalignment, and incorrect pose execution, applying continuous weighted scoring.
4. **Adaptive Voice Coaching:** Delivers low-latency verbal cues and directional hints via a prioritized, non-overlapping voice queuing engine with automated deduplication and interruptibility.
5. **Autonomous Curated Session Planning:** Automatically generates personalized multi-step practice routines tailored to user goals (Flexibility, Strength, Balance, Posture, Stress Relief), experience levels (Beginner, Intermediate, Advanced), and duration constraints.
6. **Commercial SaaS Entitlements & Subscription Management:** Includes user authentication, session telemetry persistence, weakness analytics, and an integrated payment gateway (Razorpay test/live modes) with Pro feature tier gating.

1.2 Target Audience & User Profiles

- **Independent Yoga Practitioners:** Individuals seeking guided at-home practice with immediate posture verification and hold-timer tracking.
- **Beginners & Rehabilitation Clients:** Users requiring step-by-step guidance, visual alignment cues, and proactive safety alerts.
- **Instructors & Studio Admins:** Administrators who audit pose rules, monitor student session

frequencies, and manage custom pose libraries.

1.3 Technical Stack & Core Dependencies

Backend & Core Engine (Python 3.9+)

- **Web Framework:** Flask (WSGI web server, session management, REST API endpoints).
- **Computer Vision & Kinematics:**
  - `mediapipe` (0.10.x+): High-fidelity BlazePose machine learning pipeline.
  - `opencv-python` (`cv2`): Desktop camera capture, frame preprocessing, letterbox resizing, BGR/RGB color space conversions.
  - `numpy`: Multi-dimensional vector geometry, Euclidean norms, dot products, and array transformations.
- **Database & Storage:** SQLite3 with relational foreign key enforcement and JSON string serialization for complex attributes.
- **Desktop GUI:** PySide6 (Qt for Python) for standalone offline desktop deployment.
- **Offline Text-to-Speech:** `pyttsx3` with background worker threading for desktop voice feedback.
- **Payments & Security:** `razorpay` Python SDK, `hashlib` (HMAC-SHA256 signature verification, SHA-256 password hashing).
- **Testing Framework:** `pytest` (81 automated unit and integration tests).

Frontend & Browser Engine

- **Markup & Styling:** Semantic HTML5, CSS3 with modern CSS custom properties, Warm Editorial Design System, responsive flexbox/grid, and fluid typography.
- **Client-Side Computer Vision:** MediaPipe Pose WebAssembly (`@mediapipe/camera_utils`, `@mediapipe/pose`) delivering 30+ FPS client-side inference directly inside the browser canvas.
- **Client-Side Speech Synthesis:** W3C Web Speech API (`window.speechSynthesis`) wrapped by a custom frontend `VoiceManager`.
- **Client-Side Voice Recognition:** Web Speech API (`webkitSpeechRecognition` / `SpeechRecognition`) for conversational intent processing.
- **Progressive Web App (PWA):** Service worker caching, `manifest.json`, offline capability, and mobile install prompt support.

1.4 Runtime Execution & Startup Mechanics

KI.AI operates under two distinct deployment targets:

1. Web Application Mode (Primary Commercial Target)

- **Entry Point:** `web_app.py`
- **Execution Command:**

```
python3 web_app.py
```

- **Operational Lifecycle:**

1. Loads configuration from `config/settings.py` and environment variables (`.env`).
2. Initializes database tables via `database/database.py` and seeds default poses if missing.
3. Initializes the `RazorpayClient` singleton with configured credentials or development test fallback.
4. Configures HTTP response security headers (`Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`).
5. Binds to `0.0.0.0:8081` (or port specified via environment variables / Render `PORT`).
6. Serves the Warm Editorial single-page interface (`templates/index.html`) and all REST endpoints.

2. Standalone Desktop Mode (Native Qt Application)

- **Entry Point:** `app.py`
- **Execution Command:**

```
python3 app.py
```

- **Operational Lifecycle:**

1. Instantiates `QApplication`.
2. Initializes local SQLite database connection.
3. Launches `MainWindow` (`ui/main_window.py`) presenting sidebar navigation and stacked widget views.
4. Uses native OpenCV `cv2.VideoCapture` and `vision/pose_detector.py` for desktop video streaming.
5. Uses `voice/voice_manager.py` with `pyttsx3` for native desktop audio coaching.

1.5 Repository Directory Structure

```
yoga-main/
├── .env.example                    # Environment variable configuration template
├── .gitignore                      # Git tracking ignore rules
├── KI_AI_Yoga_Assistant.spec       # PyInstaller desktop compilation specification
├── Procfile                        # Web process definition for Heroku/Render deployment
├── README.md                       # High-level product overview and quickstart guide
├── render.yaml                     # Infrastructure-as-code deployment blueprint for Render
├── requirements.txt                # Complete Python library dependencies
├── app.py                          # Standalone Desktop application entry point (PySide6)
├── build_app.py                    # PyInstaller build automation script
├── web_app.py                      # Primary Flask Web application server (38 REST endpoints)
│
├── analysis/                       # Computer Vision & Biomechanical Kinematic Engine
│   ├── angle_calculator.py         # 2D & 3D vector geometry and joint angle math
│   ├── feedback.py                 # Biomechanical feedback and verbal cue generator
│   ├── pose_classifier.py          # Active pose identification and bilateral mirror mapping
│   ├── posture_checker.py          # Rule-based joint angle verification and tolerance evaluation
│   └── score_calculator.py         # Continuous mathematical posture scoring algorithms
│
├── assets/                         # Static branding and reference visual assets
│   ├── figures/                    # Pose anatomical figure illustrations
│   ├── icons/                      # UI iconography and navigation glyphs
│   ├── logo/                       # KI.AI vector branding and logomarks
│   └── poses/                      # Visual pose demonstration reference images
│
├── coach/                          # AI Guided Practice & Dialogue Systems
│   ├── coach_engine.py             # Practice session coordinator and live feedback dispatcher
│   ├── conversation.py             # Conversational AI intent parser and dialogue engine
│   └── state_machine.py            # Finite State Machine governing practice progression
│
├── config/                         # Centralized System Configuration
│   └── settings.py                 # Global settings, thresholds, API keys, and color constants
│
├── data/                           # Local database storage directory
│   └── yoga_master.db              # Default local SQLite database file
│
├── database/                       # Relational Persistence Layer
│   ├── database.py                 # Complete Data Access Layer (CRUD, analytics, subscriptions)
│   ├── schema.sql                  # Complete relational schema (12 tables, indexes, constraints)
│   └── seed_poses.py               # Default asana seed dataset (12 core poses + Surya Namaskar)
│
├── docs/                           # Technical Architecture Documentation
│   ├── algorithms.md               # Theoretical kinematic and scoring algorithms
│   ├── architecture.md             # High-level component interactions
│   └── database.md                 # Historical database schema documentation
│
├── models/                         # Custom machine learning models and templates
```

8

```
│   └── karate_moves.json          # Reference karate kinematics template definition
│
├── payment/                       # Commercial SaaS Payment Architecture
│   └── razorpay_client.py         # Razorpay API client, test mode mocking, HMAC verification
│
├── recommendation/                # Curriculum & Session Planning Engine
│   ├── recommender.py             # User history and weakness-targeted pose recommender
│   └── session_planner.py         # Automated dynamic session curriculum planner
│
├── scripts/                       # Operational and utility scripts
│   ├── create_karate_templates.py# Script to generate martial arts biomechanical templates
│   ├── fix_db.py                  # Schema migration and database repair utility
│   ├── generate_pwa_icons.py      # PWA icon generator script
│   └── init_db.py                 # Database initialization script
│
├── static/                        # Static Web Assets
│   ├── manifest.json              # PWA Progressive Web App manifest
│   ├── sw.js                      # PWA Service Worker caching engine
│   ├── css/
│   │   └── style.css              # Complete Warm Editorial CSS Design System
│   ├── icons/                     # PWA launcher icons (192x192, 512x512)
│   └── js/
│       └── main.js                # Client-side UI manager, camera, MediaPipe, voice engine
│
├── templates/                     # Server-Rendered HTML Templates
│   ├── index.html                 # Unified Warm Editorial SPA (9 main views, 6 modals)
│   └── admin.html                 # Dedicated Administrative Management Dashboard
│
├── tests/                         # Comprehensive Automated Pytest Suite (81 Tests)
│   ├── conftest.py                # Shared test fixtures, mock databases, and landmark sets
│   ├── test_angles.py             # Unit tests for AngleCalculator vector geometry
│   ├── test_coach_engine.py       # State machine, intent parser, and coach API tests
│   ├── test_database.py           # SQLite schema, CRUD, custom templates, and analytics tests
│   ├── test_pose_rules.py         # Biomechanical rule matching and classification tests
│   ├── test_recommendation.py     # Session generator and weakness recommendation tests
│   ├── test_scoring.py            # Mathematical posture scoring and decay curve tests
│   ├── test_subscription_payment.py # Payment verification, HMAC, and subscription tests
│   ├── test_vision_pipeline.py    # Landmark validation, symmetry, and frame analysis tests
│   └── test_voice_manager.py      # Voice priority queue, cooldowns, and deduplication tests
│
├── ui/                            # Standalone PySide6 Desktop GUI Components
│   ├── admin_window.py            # Desktop administrative panel
│   ├── analytics_window.py        # Desktop historical analytics dashboard
│   ├── camera_window.py           # Desktop OpenCV live camera viewfinder
│   ├── coach_window.py            # Desktop conversational coaching panel
│   ├── main_window.py             # Desktop root window and sidebar layout
│   ├── pose_window.py             # Desktop pose library explorer
```

9

```
│   ├── session_window.py          # Desktop guided practice player
│   └── components/                # Desktop reusable Qt widgets (cards, gauges, meters)
│
├── vision/                        # Desktop Video Capture & MediaPipe Binding
│   ├── camera.py                  # Thread-safe OpenCV video capture wrapper
│   ├── landmarks.py               # MediaPipe landmark indices and body segment definitions
│   └── pose_detector.py           # Python MediaPipe Pose detector wrapper for OpenCV
│
└── voice/                         # Native Audio & Text-to-Speech Engine
    ├── tts_engine.py              # Thread-safe pyttsx3 worker thread for desktop
    └── voice_manager.py           # Centralized voice queuing, priority, and cooldown coordinator
```

SECTION 2 — COMPLETE FILE AND FOLDER EXPLANATION

2.1 Top-Level Entry Points & Configurations

File Path	Primary Responsibility	Key Classes & Functions	Upstream / Downstream Dependencies
`web_app.py`	Web application server; initializes Flask, registers 38 REST endpoints, manages sessions, coordinates computer vision payloads, and serves the UI.	`app`, `login_required()`, `pro_required()`, `add_security_headers()`	Imports from `analysis/`, `coach/`, `database/`, `payment/`, `recommendation/`, `voice/`. Serves `templates/index.html`.
`app.py`	Desktop application entry point; initializes PySide6 `QApplication`, applies dark theme, launches `MainWindow`.	`main()`, `setup_environment()`	Imports `ui.main_window.MainWindow`, `database.database.Database`.
`config/settings.py`	Global configuration constants, scoring thresholds, tolerance defaults, color hex codes,	`CAMERA_CONFIG`, `SCORING_THRESHOLDS`, `COLORS`, `JOINT_PAIRS`, `DEFAULT_TOLERANCE`	Referenced throughout all Python modules in `analysis/`, `vision/`, and `database/`.

File Path	Primary Responsibility	Key Classes & Functions	Upstream / Downstream Dependencies
	and environment variable loaders.		
`requirements.txt`	Defines pinned/ compatible runtime dependencies.	`Flask`, `mediapipe`, `opencv-python`, `numpy`, `PySide6`, `pyttsx3`, `razorpay`, `pytest`	Used by pip and cloud deployment buildpacks.
`build_app.py`	PyInstaller build script for creating single-file or standalone desktop binaries.	`build()`	Wraps `KI_AI_Yoga_Assistant.spec` and compiles `app.py`.

2.2 Analysis Engine (`analysis/`)

This directory houses the deterministic kinematic and mathematical reasoning modules.

1. `analysis/angle_calculator.py`

- **Responsibility:** Computes 2D and 3D joint angles using vector algebra and trigonometric dot products.
- **Key Methods:**
  - `calculate_angle(point_a, point_b, point_c, use_3d=False)`: Computes the inner angle at vertex `point_b` formed by vectors \vec{BA} and \vec{BC}. Clamps cosine values to [-1.0, 1.0] before applying arccos to prevent floating-point domain errors.
  - `calculate_angle_with_vertical(point_a, point_b)`: Computes segment angle relative to the true vertical axis (0, -1).
  - `calculate_angle_with_horizontal(point_a, point_b)`: Computes segment angle relative to the horizontal axis (1, 0).
  - `extract_all_angles(landmarks)`: Extracts all primary joint angles (left/right elbow, shoulder, hip, knee, ankle, spine) from a landmark dictionary or list.

2. `analysis/pose_classifier.py`

- **Responsibility:** Identifies which pose the user is performing by comparing measured angles

against stored pose templates, supporting bilateral mirror orientation.

- **Key Methods:**
  - `identify_pose(measured_angles, candidate_poses, active_pose_name=None, threshold=15.0)`: Compares measured angles against candidate poses. Prioritizes the active pose if within threshold.
  - `_calculate_pose_distance(measured, reference, mirror=False)`: Computes the mean absolute error across common joints, automatically translating joint names across the bilateral axis (`left_knee` ≤ftrightarrow `right_knee`) when `mirror=True`.

3. `analysis/posture_checker.py`

- **Responsibility:** Validates a user's joint angles against database pose rules (`pose_rules` table), flagging out-of-tolerance joints.
- **Key Methods:**
  - `check_posture(measured_angles, pose_rules)`: Evaluates measured angles against `target_angle` and `tolerance`. Computes deviations, flags incorrect joints, and compiles feedback messages.
  - `check_pose_by_name(measured_angles, pose_name)`: Convenience wrapper querying rules from the database and executing posture evaluation.

4. `analysis/score_calculator.py`

- **Responsibility:** Computes overall and per-joint accuracy scores using linear decay within tolerance and steep exponential penalty decay beyond tolerance.
- **Key Methods:**
  - `calculate_joint_score(measured, target, tolerance)`: Returns 100.0 at exact target, decays to 90.0 at tolerance boundary, and decays toward 0.0 beyond tolerance.
  - `calculate_overall_score(joint_scores, weights=None)`: Computes weighted average score across all active joints.
  - `get_score_level(score)`: Maps numerical scores to qualitative levels (`Excellent` ≥ 90, `Good` ≥ 75, `Needs Work` ≥ 60, `Poor` < 60).

5. `analysis/feedback.py`

- **Responsibility:** Generates human-understandable verbal and visual alignment cues from joint deviations.
- **Key Methods:**

- `generate_feedback(joint_checks, pose_name)`: Ranks joint deviations by weight and severity, returning concise corrective instructions (e.g., *"Bend your left knee more"*, *"Straighten your spine"*).

2.3 Computer Vision & Landmark Extraction (`vision/`)

1. `vision/landmarks.py`

- **Responsibility:** Maps BlazePose landmark integer indices (0 to 32) to human-readable names and anatomical groups.
- **Key Constants:**
  - `NOSE = 0`, `LEFT_SHOULDER = 11`, `RIGHT_SHOULDER = 12`, `LEFT_ELBOW = 13`, `RIGHT_ELBOW = 14`, `LEFT_WRIST = 15`, `RIGHT_WRIST = 16`, `LEFT_HIP = 23`, `RIGHT_HIP = 24`, `LEFT_KNEE = 25`, `RIGHT_KNEE = 26`, `LEFT_ANKLE = 27`, `RIGHT_ANKLE = 28`.
  - `POSE_CONNECTIONS`: Tuple pairs defining bone connections for skeleton rendering.
  - `KEY_LANDMARKS`: Subsets of essential landmarks required for full-body tracking verification.

2. `vision/pose_detector.py`

- **Responsibility:** Desktop MediaPipe Pose wrapper for processing OpenCV frames.
- **Key Methods:**
  - `find_pose(frame, draw=True)`: Processes BGR frames through MediaPipe Pose and annotates skeleton connections.
  - `find_positions(frame)`: Extracts normalized (x, y, z, \text{visibility}) landmark coordinates.
  - `is_body_visible(landmarks)`: Returns boolean indicating whether critical landmarks exceed visibility confidence thresholds.

3. `vision/camera.py`

- **Responsibility:** Thread-safe OpenCV camera video stream grabber for desktop execution.
- **Key Methods:**
  - `start()`, `stop()`, `get_frame()`: Threaded capture loop preventing UI lockup during frame decoding.

2.4 Coaching & Session Orchestration (`coach/` & `recommendation/`)

1. `coach/state_machine.py`

- **Responsibility:** Finite State Machine managing the lifecycle of an exercise within a guided practice session.
- **Key States:** `IDLE`, `ENTERING_POSE`, `ALIGNING`, `HOLDING`, `COMPLETED`, `PAUSED`.
- **Key Methods:**
  - `transition_to(new_state)`: Validates state transitions.
  - `update_hold(dt, is_correct)`: Increments hold timer only when `is_correct=True`. Automatically transitions to `COMPLETED` when elapsed hold time reaches target.

2. `coach/coach_engine.py`

- **Responsibility:** Main coordinator for guided live practice; evaluates incoming frames, manages the state machine, and triggers voice coaching events.
- **Key Methods:**
  - `load_exercise(pose_name, target_hold_seconds)`: Prepares the engine for a specific pose.
  - `evaluate_frame(landmarks)`: Full evaluation pipeline: checks body visibility, calculates angles, checks posture, computes score, advances hold timer, and dispatches verbal cues via `VoiceManager`.

3. `coach/conversation.py`

- **Responsibility:** Conversational intent parser and coach response generator.
- **Key Methods:**
  - `process_query(user_text, current_session_context)`: Parses user intent (pain/safety alerts, skip pose, shorten session, explain score, general question) using regex and keyword heuristics, returning tailored coaching dialogue.

4. `recommendation/session_planner.py`

- **Responsibility:** Dynamic multi-exercise routine generator.
- **Key Methods:**
  - `generate_session(goal, level, duration_minutes, focus_area=None)`: Curates a

balanced 3-phase session sequence (Warmup, Main Practice, Cooldown) selecting appropriate poses, target hold durations, and rest intervals.

5. `recommendation/recommender.py`

- **Responsibility:** Historical weakness analyzer and targeted recommendation engine.
- **Key Methods:**
  - `get_recommendations_for_user(user_id, limit=5)`: Analyzes historical joint accuracy from `practice_sessions` and `feedback` tables, recommending poses that directly train the user's lowest-scoring joints.

2.5 Audio & Voice Management (`voice/`)

1. `voice/voice_manager.py`

- **Responsibility:** Centralized voice coaching coordinator handling priority queuing, deduplication, cooldown windows, and manual interruptions.
- **Key Classes:** `VoiceEvent`, `VoiceManager`
- **Key Priorities:** `SAFETY = 100`, `CRITICAL = 80`, `CORRECTION = 60`, `PRAISE = 40`, `INFO = 20`.
- **Key Methods:**
  - `request_speech(text, priority, event_type, target_pose, bypass_cooldown=False)`: Filters identical messages within a cooldown period, manages priority queue size, and dispatches speech.
  - `stop_immediately()`: Cancels current speech, clears queue, and halts playback.
  - `mute()` / `unmute()`: Suppresses non-safety audio output.
  - `interrupt_for_listening()`: Pauses speech when the user activates microphone input.

2. `voice/tts_engine.py`

- **Responsibility:** Native desktop `pyttsx3` text-to-speech worker thread.
- **Key Methods:** `speak_async(text)`, `stop()`.

2.6 Persistence & Payments (`database/` & `payment/`)

1. `database/database.py`

- **Responsibility:** Data access layer encapsulating all SQLite operations.
- **Key Capabilities:**
  - User authentication and session persistence (`create_user`, `verify_user`, `get_user_by_id`).
  - Pose and rule retrieval (`get_all_poses`, `get_pose_by_name`, `get_pose_rules`).
  - Practice history and analytics queries (`save_practice_session`, `get_user_analytics`, `get_joint_accuracy_history`).
  - Guided session logging (`save_guided_session_log`, `get_user_guided_sessions`).
  - Subscription and payment tracking (`create_subscription`, `get_user_subscription`, `record_payment`, `update_payment_status`).

2. `payment/razorpay_client.py`

- **Responsibility:** Integration wrapper for Razorpay payment gateway with automatic test-mode fallback.
- **Key Methods:**
  - `create_order(amount_inr, receipt, notes)`: Generates a standard Razorpay order ID (or mock `order_test_*` in dev mode).
  - `verify_payment_signature(razorpay_order_id, razorpay_payment_id, razorpay_signature)`: Computes HMAC-SHA256 signature using `key_secret` and performs constant-time comparison against incoming webhook/client signatures.
  - `get_public_config()`: Safely returns public key ID without leaking secret.

SECTION 3 — FRONTEND ARCHITECTURE AND USER INTERFACE

3.1 Design Philosophy & The "Warm Editorial" Design System

The frontend of KI.AI is engineered around an intentional **Warm Editorial Design System** (`static/css/style.css` and `templates/index.html`). Rejecting the cold, hyper-saturated neon aesthetics common in fitness gadgets, KI.AI adopts a mindful, high-end wellness aesthetic reminiscent of luxury lifestyle publications and physical yoga studios.

CSS Custom Property Tokens

```
:root {
  /* Core Color Palette */
  --sand-50: #FAF8F5;           /* Primary application canvas background */
  --sand-100: #F4EFEB;          /* Card, container, and elevated surface backgrounds */
  --cream: #EDE5DC;             /* Subtle borders, divider rules, and input backgrounds */
  --espresso: #1A1615;          /* Deep primary typography, headings, high-contrast badges */
  --charcoal: #2E2927;          /* Body copy, labels, secondary headers */
  --text-muted: #7A726D;        /* Subtext, metadata, breadcrumbs, placeholder text */
  --terracotta: #C85A32;        /* Accent highlight, primary CTA buttons, active states */
  --sage: #5B7065;              /* Secondary accents, calm indicators, wellness highlights */

  /* Semantic Status Indicators */
  --status-success: #2E7D32;    /* Correct posture, target hold active, payment verified */
  --status-warning: #ED6C02;    /* Joint misalignment, partial posture, adjustment required */
  --status-error: #D32F2F;      /* Wrong pose detected, tracking lost, payment failed */
  --status-info: #0288D1;       /* Instructional hints, system announcements */

  /* Typography Scale */
  --font-serif: 'Playfair Display', Georgia, serif;    /* Editorial headlines, pose titles */
  --font-sans: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif; /* Body copy, UI, m

  /* Elevation & Geometry */
  --radius-sm: 8px;
  --radius-md: 16px;
  --radius-lg: 24px;
  --shadow-sm: 0 2px 8px rgba(26, 22, 21, 0.04);
  --shadow-md: 0 8px 24px rgba(26, 22, 21, 0.08);
  --shadow-lg: 0 16px 40px rgba(26, 22, 21, 0.12);
```

```
}
```

3.2 Web Single-Page Application (SPA) Navigation & View Management

The primary web client (`templates/index.html`) operates as an unbundled, dependency-free vanilla JavaScript Single-Page Application. Page switches occur instantaneously without full-browser refreshes by manipulating the `.active` class across nine dedicated view sections:

```
function switchView(viewName) {
  // Hide all sections
  document.querySelectorAll('.view-section').forEach(el => el.classList.remove('active'));
  // Update header navigation active styling
  document.querySelectorAll('.nav-link').forEach(el => el.classList.remove('active'));

  // Activate selected section
  const target = document.getElementById(`view-${viewName}`);
  if (target) target.classList.add('active');

  // Trigger contextual view-initialization logic
  if (viewName === 'home') loadHomeView();
  else if (viewName === 'library') loadPoseLibrary();
  else if (viewName === 'progress') loadProgressDashboard();
  else if (viewName === 'programs') loadProgramsCatalog();
  else if (viewName === 'pricing') loadPricingDetails();
  else if (viewName === 'preferences') loadUserPreferences();

  // Enforce camera privacy shutdown if exiting practice view
  if (viewName !== 'practice' && CameraManager.state !== CameraState.IDLE) {
    CameraManager.stopCamera();
  }
}
```

3.3 Detailed Inventory of the 9 Main Web Views

1. Home Dashboard (`#view-home`)

- **Visual Composition:** Editorial hero welcoming the user by name with personalized wellness

greeting, daily streak indicator, "Today's Guided Flow" recommendation card, fast-launch button, and a curated list of popular postures.

- **Key Interactivity:**
    - One-click trigger for the daily AI session generator (`startDailySession()`).
    - Quick-start pose shortcuts that pre-configure single-pose practices.
    - Quick links to subscription plans and posture progress.

2. Session Intro (`#view-session-intro`)

- **Visual Composition:** Pre-practice sequence staging room. Displays the dynamic multi-step routine created by `session_planner.py`, listing the estimated duration, total exercises, target focus area, and sequential card timeline (Warmup → Active Poses → Cooldown).
- **Key Interactivity:**
    - "Begin Guided Practice" button triggering the Camera Privacy Modal.
    - Individual exercise preview modals showing target alignment points and benefits.
    - "Adjust Duration or Focus" shortcut redirecting to Preferences.

3. Live Practice Studio (`#view-practice`)

- **Visual Composition:** High-performance, full-viewport practice environment. Contains:
    - Dual-layer viewport: HTML5 `<video id="webcam">` overlaid with high-DPI HTML5 `<canvas id="canvas-overlay">`.
    - Floating Real-Time Head-Up Display (HUD) showing active pose name, Sanskrit translation, hold timer progress bar, accuracy gauge (0–100%), and body tracking badge (`NO PERSON`, `PARTIAL BODY`, `GOOD TRACKING`).
    - Joint Correction Telemetry Panel: Live red/yellow diagnostic pill flags indicating exact misaligned joints (e.g., `Right Knee: 112° (Target: 90°)`).
    - Floating AI Voice Coach Bubble: Live speech transcription indicator showing the coach's verbal cues.
    - Studio Controls: Emergency "Stop Practice", Voice Mute toggle, Speech Feedback Cooldown toggle, and Push-to-Talk microphone button for conversational queries.
- **Key Interactivity:** Real-time MediaPipe Pose loop running at 30 FPS, continuous frame analysis via REST API, hold-timer advancement, and step auto-advancement.

4. Session Complete Summary (`#view-session-complete`)

- **Visual Composition:** Post-practice celebration view presenting session statistics: total practice time, average posture accuracy percentage, total completed poses, calories burned estimate,

and joint mastery ratings.
- **Key Interactivity:**
    - "Save to Practice Log" button persisting completion data to `guided_session_logs`.
    - Personalized coach summary note explaining top strengths and areas for mobility improvement.
    - "Return to Home" button.

5. Guided Programs Catalog (`#view-programs`)

- **Visual Composition:** Multi-week structured curriculum cards categorized by discipline:
    - *7-Day Morning Vitality Flow* (Beginner)
    - *14-Day Postural Correction & Spine Alignment* (All Levels)
    - *21-Day Core & Asymmetric Balance Mastery* (Intermediate)
    - *Martial Kinematics & Karate Stance Primer* (Beta / Pro)
- **Key Interactivity:** "Enroll & Start Today" buttons that set active user program and initialize customized daily routines.

6. Posture Progress & Analytics (`#view-progress`)

- **Visual Composition:** Comprehensive historical telemetry dashboard. Renders:
    - Weekly practice frequency bar charts.
    - Overall accuracy trendline over time.
    - Joint-by-joint mobility radar/heatmap: identifies anatomical areas with recurring deviations (e.g., hip flexion vs. knee extension).
    - Chronological practice log history table with date, pose, duration, and score.
- **Key Interactivity:** Data dynamically fetched via `/api/user/analytics` and rendered via responsive SVG charts.

7. Pose & Exercise Library (`#view-library`)

- **Visual Composition:** Filterable visual encyclopedia of all 12+ built-in asanas plus Surya Namaskar sequences.
- **Filter Controls:** Category buttons (Standing, Balance, Inversion, Seated, Restorative) and difficulty pills (Beginner, Intermediate, Advanced).
- **Card Data:** Anatomical illustration, English and Sanskrit names, target joint angles, therapeutic benefits, and "Practice Now" launch button.

8. Pricing & SaaS Pro Plans (`#view-pricing`)

- **Visual Composition:** Commercial SaaS tier selector contrasting Free Tier vs. Pro Subscription:

    - *Free Tier (₹0):* 3 basic poses, standard accuracy score, community access.
    - *Pro Monthly (₹799/mo):* Full 12+ pose library, Surya Namaskar, AI Voice Coach with dynamic real-time speech, unlimited guided sessions, advanced joint telemetry, and priority beta access to Karate AI.
    - *Pro Annual (₹5,999/yr):* All Pro features + 37% annual savings and customized 1-on-1 AI routine generation.
- **Key Interactivity:** "Upgrade to Pro" triggers Razorpay checkout modal (`PaymentManager.startCheckout()`).

9. Personal Preferences & Settings (`#view-preferences`)

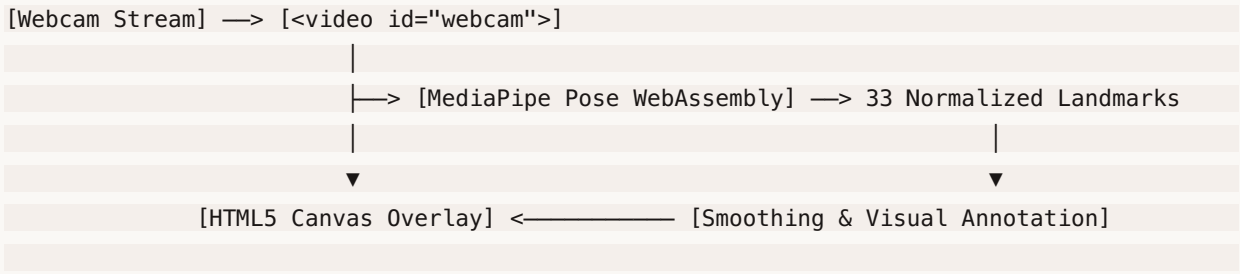
- **Visual Composition:** Onboarding and settings configuration form:
    - Primary Practice Goal (General Fitness, Flexibility, Stress Relief, Strength).
    - Experience Level (Beginner, Intermediate, Advanced).
    - Target Practice Duration (10, 15, 20, 30 minutes).
    - Focus Area (Full Body, Hips & Hamstrings, Spine & Back, Core Stability).
    - AI Voice Coach toggle and speech rate slider.
- **Key Interactivity:** Direct synchronization with backend `/api/user/preferences`.

3.4 Modals, Overlays, and Interactive Dialogs

1. **Camera Privacy Modal (`#modal-camera-privacy`):**
    - **Purpose:** Strict privacy reassurance modal explaining that raw video frames are processed exclusively on the user's local machine and never streamed or stored on external servers.
    - **Action:** Triggers `confirmStartPracticeCamera()`, invoking `navigator.mediaDevices.getUserMedia`.
2. **Payment Checkout Modal (`#modal-payment-checkout`):**
    - **Purpose:** Displays Razorpay checkout interface. In production, mounts Razorpay iframe; in test/dev mode, simulates order generation and triggers server-side HMAC signature verification.
3. **Karate AI Expansion Modal (`#karate-modal`):**
    - **Purpose:** Teaser and beta waitlist modal for the upcoming martial arts stance analysis extension (Zenkutsu Dachi, Kiba Dachi).
4. **Voice Coach Listening Overlay:**
    - **Purpose:** Visual waveform overlay activated when the user holds the microphone button to speak to the coach, displaying transcribed queries in real time.

3.5 Real-Time Canvas Rendering Pipeline & Visual Skeleton HUD

The browser practice studio (`#view-practice`) uses a synchronized double-buffered rendering pipeline combining HTML5 `<video>` and `<canvas>`:



Key Visual Skeleton Design Rules:

1. **Dynamic Color-Coding:**
    - `Green (#2E7D32)`: Joint angle is within codified tolerance (≥ 75% score).
    - `Amber (#ED6C02)`: Joint angle is misaligned, requiring minor adjustment (60–74% score).
    - `Red (#D32F2F)`: Joint is severely out of position (< 60% score).
2. **Bone Rendering:** Draws smoothed connections between adjacent landmark pairs (`LEFT_SHOULDER` to `LEFT_ELBOW`, `LEFT_ELBOW` to `LEFT_WRIST`, etc.) with 4px stroke width and rounded joint caps.
3. **Target Angle Arc Overlay:** Computes 2D angle arcs directly over the user's live video feed, rendering the current measured angle in degrees alongside an arrow pointing in the direction of the target angle.

3.6 Frontend Audio & Voice Interaction (`window.VoiceManager`)

To eliminate the common bug of infinite overlapping speech synthesis loops in browser environments, the frontend implements a robust `window.VoiceManager` client singleton:

- **State Flags:** `isSpeaking`, `isMuted`, `lastSpokenText`, `lastSpokenTime`.
- **Deduplication Cooldown:** Identical feedback phrases (e.g., *"Straighten your back"*) are suppressed if triggered within 6.0 seconds of their prior utterance.

- **Speech Priority Handling:** Safety alerts and pose completion announcements preempt standard posture adjustments.
- **Instant Speech Cancellation:** Invoking `VoiceManager.stopImmediately()` calls `window.speechSynthesis.cancel()` and purges internal speech queues immediately when the user pauses or changes postures.

3.7 Desktop PySide6 GUI Architecture (`ui/` Directory)

For native offline desktop environments, the application includes a full Qt/PySide6 implementation:

- `ui/main_window.py`: Root application container featuring a collapsible left sidebar, custom titlebar, and `QStackedWidget` managing sub-windows.
- `ui/camera_window.py`: Custom `QWidget` embedding an OpenCV camera thread, rendering frames via `QImage` and `QPixmap` on a high-refresh `QLabel`.
- `ui/session_window.py`: Qt practice controller managing countdown timers, sound alerts, and pose transitions.
- `ui/coach_window.py`: Native conversational dialogue panel featuring a chat message bubble layout and speech toggle.
- `ui/analytics_window.py`: QtCharts/Matplotlib historical analytics view displaying weekly practice bars and accuracy distribution.
- `ui/pose_window.py`: Grid of pose cards with image thumbnails and instruction popovers.
- `ui/admin_window.py`: Administrative table view for inspecting SQLite records and editing pose rules.

SECTION 4 — BACKEND ARCHITECTURE AND API ROUTES

4.1 Server Overview & Middleware Pipeline

The KI.AI backend is built on **Flask** (`web_app.py`), orchestrating the data layer (`database/database.py`), biomechanical analysis engines (`analysis/`), guided coaching state machines (`coach/`), voice queues (`voice/`), and commercial payment processing (`payment/`).

Security Headers & Middleware

Every HTTP response sent by `web_app.py` passes through the `@app.after_request` middleware handler `add_security_headers()`, enforcing strict production hardening:

- `X-Content-Type-Options: nosniff`: Prevents MIME-type sniffing attacks.
- `X-Frame-Options: SAMEORIGIN`: Defends against clickjacking.
- `Referrer-Policy: strict-origin-when-cross-origin`: Minimizes referrer leakage.
- `Permissions-Policy: camera=(self), microphone=(self)`: Restricts hardware sensor access strictly to the KI.AI origin.
- `Content-Security-Policy`: Permits necessary CDNs (MediaPipe WebAssembly, Google Fonts, Razorpay checkout frame) while preventing untrusted script execution.

4.2 Comprehensive REST API Endpoint Inventory (38 Endpoints)

The 38 endpoints in `web_app.py` are grouped into nine functional domains:

```
├── 1. Core Web Views & Assets (3 endpoints)
├── 2. Authentication & User Management (2 endpoints)
├── 3. Pose Catalog & Sequences (2 endpoints)
├── 4. Computer Vision & Posture Analysis (2 endpoints)
├── 5. Practice History & User Analytics (4 endpoints)
├── 6. Dynamic Programs & Curriculum (3 endpoints)
├── 7. Guided Sessions & Conversational Coach (5 endpoints)
├── 8. Voice & Audio Engine Controls (8 endpoints)
└── 9. Subscriptions, Payments & PWA Manifest (9 endpoints)
```

Group 1: Core Web Views & Static Assets

1. `GET /`

- **Purpose:** Serves the primary Warm Editorial single-page interface.
- **Auth Required:** No.
- **Response:** Rendered HTML template (`templates/index.html`).

2. `GET /admin`

- **Purpose:** Serves the administrative database and pose management console.
- **Auth Required:** No (in development; accessible directly).
- **Response:** Rendered HTML template (`templates/admin.html`).

3. `GET /assets/<path:filename>`

- **Purpose:** Serves static figures, pose photographs, and branding assets from the `/assets` directory.
- **Parameters:** `filename` (string path).
- **Response:** Binary image file (`image/jpeg`, `image/png`, `image/svg+xml`).

Group 2: Authentication & User Management

4. `POST /api/login`

- **Purpose:** Authenticates user credentials.
- **Auth Required:** No.
- **Request Body (JSON):**

```
{
  "email": "ashik@ki.ai",
  "password": "securepassword123"
}
```

- **Processing:** Hashes password with SHA-256 and queries `users` table via `db.verify_user()`.
- **Response (Success - 200):**

```
{
  "success": true,
  "user": {
    "id": 1,
    "name": "Ashik",
    "email": "ashik@ki.ai",
    "goal": "General Fitness",
    "level": "Beginner",
    "is_pro": false
  }
}
```

- **Error Response (401):** `{"success": false, "error": "Invalid email or password"}`.

5. `POST /api/register`

- **Purpose:** Creates a new user profile.
- **Auth Required:** No.
- **Request Body (JSON):**

```
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "password123",
  "goal": "Flexibility",
  "level": "Intermediate",
  "age": 28
}
```

- **Processing:** Validates unique email, hashes password with SHA-256, and inserts record into `users` table via `db.create_user()`.
- **Response (Success - 201):** `{"success": true, "user_id": 2}`.
- **Error Response (400):** `{"success": false, "error": "Email already exists or invalid fields"}`.

Group 3: Pose Catalog & Sequences

6. `GET /api/poses`

- **Purpose:** Retrieves the full catalog of available yoga poses with target joint rules.

- **Auth Required:** No.
- **Response (200):**

```
[
  {
    "id": 1,
    "name": "Tadasana",
    "sanskrit_name": "Tadasana",
    "difficulty": "Beginner",
    "category": "Standing",
    "description": "Mountain Pose promotes steady balance and posture.",
    "benefits": "Improves posture, strengthens thighs and ankles.",
    "target_angles": { "left_knee": 180, "right_knee": 180, "spine": 180 }
  }
]
```

7. `GET /api/surya`

- **Purpose:** Retrieves the structured 12-step Surya Namaskar (Sun Salutation) sequence with step numbers, breathing patterns, and hold targets.
- **Auth Required:** No.
- **Response (200):** JSON array containing the 12 sequential postures with breathing phase cues (`Inhale`, `Exhale`, `Hold`).

Group 4: Computer Vision & Posture Analysis

8. `POST /api/analyze_posture`

- **Purpose:** Standalone computer vision endpoint for single-pose verification. Analyzes a raw set of 33 landmarks against a target pose.
- **Auth Required:** No.
- **Request Body (JSON):**

```
{
  "landmarks": [ { "x": 0.52, "y": 0.21, "z": -0.1, "visibility": 0.98 }, ... ],
  "pose_name": "Tadasana",
  "user_id": 1
}
```

- **Processing:**

1. Validates tracking confidence across landmarks (`NO_PERSON`, `PARTIAL_BODY`, `GOOD_TRACKING`).
2. Computes 2D/3D joint angles using `AngleCalculator`.
3. Checks bilateral symmetry via `PoseClassifier`.
4. Evaluates biomechanical joint deviations using `PostureChecker`.
5. Computes overall and joint scores using `ScoreCalculator`.
6. Generates verbal feedback cues via `FeedbackGenerator`.

- **Response (200):**

```
{
  "tracking_status": "GOOD_TRACKING",
  "identified_pose": "Tadasana",
  "pose_status": "CORRECT",
  "overall_score": 94.2,
  "score_level": "Excellent",
  "joint_angles": { "left_knee": 178.4, "right_knee": 179.1, "spine": 176.8 },
  "joint_scores": { "left_knee": 98.4, "right_knee": 99.1, "spine": 96.8 },
  "incorrect_joints": [],
  "feedback": "Perfect form! Hold the mountain pose.",
  "is_stable": true
}
```

9. `POST /api/session/evaluate_frame`

- **Purpose:** High-throughput streaming frame analysis endpoint designed specifically for live guided sessions. Directly interacts with the `CoachEngine` state machine.
- **Auth Required:** No.
- **Request Body (JSON):**

```
{
  "landmarks": [ ... ],
  "expected_pose": "Warrior II",
  "target_hold": 20,
  "current_hold": 12.5,
  "user_id": 1
}
```

- **Processing:** Integrates posture analysis with the state machine. Advances the hold timer only if `pose_status == "CORRECT"` and stability checks pass. Automatically queues voice coaching cues via `VoiceManager` with debounced cooldowns.
- **Response (200):** Returns pose analysis metrics plus session state: `current_state`, `hold_seconds_remaining`, `is_holding`, and `voice_event` payload.

Group 5: Practice History & User Analytics

10. `POST /api/save_session`

- **Purpose:** Saves individual practice session telemetry to the `practice_sessions` table.
- **Request Body (JSON):**

```
{
  "user_id": 1,
  "pose_name": "Tadasana",
  "duration_seconds": 45,
  "accuracy_score": 91.5,
  "feedback_logs": [ { "joint": "left_knee", "deviation": 4.5 } ]
}
```

- **Response (200):** `{"success": true, "session_id": 104}`.

11. `GET /api/history/<int:user_id>`

- **Purpose:** Returns the past 50 single-pose practice session records for the user.
- **Response (200):** JSON array of session logs with timestamps, durations, and scores.

12. `GET /api/stats/<int:user_id>`

- **Purpose:** Computes aggregate stats: total practice time, average accuracy, total completed sessions, and top practiced poses.
- **Response (200):** Aggregate statistics object.

13. `GET /api/progress/<int:user_id>`

- **Purpose:** Detailed historical progress breakdown including weekly practice frequency and per-joint accuracy averages for radar visualization.
- **Response (200):** Structured progress and joint accuracy metrics.

Group 6: Dynamic Programs & Curriculum

14. `GET /api/programs`

- **Purpose:** Returns all multi-week guided curriculum programs.
- **Response (200):** Array of program blueprints with titles, goals, durations, and level tags.

15. `GET /api/program/<level>`

- **Purpose:** Retrieves programs filtered by difficulty level (`Beginner`, `Intermediate`, `Advanced`).
- **Response (200):** Filtered program objects.

16. `GET /api/daily-session/<int:user_id>`

- **Purpose:** Dynamically plans today's customized yoga session based on the user's stored preferences and historical joint weaknesses using `SessionPlanner`.
- **Response (200):** Complete session sequence containing warmups, core postures, target hold durations, and cooldowns.

Group 7: Guided Sessions & Conversational Coach

17. `POST /api/session/start`

- **Purpose:** Initializes a guided multi-step practice session and creates a tracking record in `session_plans`.
- **Request Body (JSON):**

```
{
  "user_id": 1,
  "goal": "Strength",
  "duration_minutes": 15,
  "level": "Intermediate",
  "focus_area": "Core Stability"
}
```

- **Response (200):** Returns `plan_id` and the generated sequence of exercises.

18. `POST /api/session/complete`

- **Purpose:** Concludes a guided session, persisting final duration, overall completion score, and notes to `guided_session_logs`.

- **Request Body (JSON):**

```
{
  "plan_id": 12,
  "user_id": 1,
  "total_duration": 900,
  "average_score": 88.4,
  "completed_exercises": 6
}
```

- **Response (200):** `{"success": true, "log_id": 8}`.

19. `GET /api/user/preferences/<int:user_id>`

- **Purpose:** Retrieves user preferences (goal, level, duration, focus, voice toggle).
- **Response (200):** Preference record object.

20. `POST /api/user/preferences`

- **Purpose:** Updates user onboarding and practice settings.
- **Request Body (JSON):** Key-value dictionary of updated preferences.
- **Response (200):** `{"success": true}`.

21. `POST /api/coach/message`

- **Purpose:** Processes natural language queries sent to the AI Coach during or outside practice.
- **Request Body (JSON):**

```
{
  "user_id": 1,
  "message": "My lower back hurts during Cobra pose",
  "current_pose": "Bhujangasana"
}
```

- **Processing:** Analyzes text using `ConversationalCoach.process_query()`. Recognizes pain/safety flags, requests to skip, or explanations of alignment. Logs dialogue to `coach_conversations`.
- **Response (200):**

```
{
  "reply": "Please ease off immediately. Lower your chest slightly and keep your elbows gently
  "action": "SAFETY_ALERT",
  "intent": "PAIN_ALERT"
```

```
}
```

22. `GET /api/coach/history/<int:user_id>`

- **Purpose:** Retrieves chronological dialogue logs between the user and the AI Coach.
- **Response (200):** Array of message objects (`user_message`, `coach_reply`, `timestamp`).

Group 8: Voice & Audio Engine Controls

23. `POST /api/voice/stop`

- **Purpose:** Immediately cancels all active speech and flushes queued audio feedback.
- **Response (200):** `{"success": true, "status": "stopped"}`.

24. `POST /api/voice/resume`

- **Purpose:** Resumes the voice coaching queue following a manual pause.
- **Response (200):** `{"success": true, "status": "resumed"}`.

25. `POST /api/voice/mute`

- **Purpose:** Mutes voice coaching output (safety warnings remain permitted if critical).
- **Response (200):** `{"success": true, "is_muted": true}`.

26. `POST /api/voice/unmute`

- **Purpose:** Restores audio speech output.
- **Response (200):** `{"success": true, "is_muted": false}`.

27. `POST /api/voice/toggle_mute`

- **Purpose:** Toggles the voice coach mute state.
- **Response (200):** `{"success": true, "is_muted": boolean}`.

28. `POST /api/voice/interrupt`

- **Purpose:** Notifies the voice engine that the user is activating speech-to-text input, temporarily pausing speech.

- **Response (200):** `{"success": true, "status": "interrupted_for_listening"}`.

29. `GET /api/voice/status`

- **Purpose:** Inspects current voice queue metrics (speaking state, queue depth, cooldown timestamps).
- **Response (200):** Voice queue diagnostic state object.

30. `POST /api/voice/speak`

- **Purpose:** Dispatches an explicit verbal message to the voice queue with priority and deduplication.
- **Request Body (JSON):** `{"text": "Hold strong for five more seconds", "priority": 60, "event_type": "PRAISE"}`.
- **Response (200):** `{"success": true, "spoken": boolean}`.

Group 9: Subscriptions, Payments & PWA Manifest

31. `GET /manifest.json`

- **Purpose:** Serves the Web App Manifest for mobile installation and PWA capabilities.
- **Response (200):** JSON manifest defining app name, background colors, display mode (`standalone`), and icon paths.

32. `GET /api/payment/config`

- **Purpose:** Delivers public payment configuration (Razorpay Key ID and pricing tier catalog) without exposing secret credentials.
- **Response (200):**

```
{
  "key_id": "rzp_test_KI_AI_DEMO_KEY",
  "is_test_mode": true,
  "plans": {
    "pro_monthly": { "amount": 79900, "currency": "INR", "name": "KI.AI Pro Monthly" },
    "pro_annual": { "amount": 599900, "currency": "INR", "name": "KI.AI Pro Annual" }
  }
}
```

33. `POST /api/payment/create-order`

- **Purpose:** Creates a Razorpay order entity for checkout initialization.
- **Request Body (JSON):**

```
{
  "user_id": 1,
  "plan_id": "pro_monthly"
}
```

- **Processing:** Invokes `RazorpayClient.create_order()`, logs an initial pending payment record in the `payments` table.
- **Response (200):**

```
{
  "success": true,
  "order_id": "order_test_948194",
  "amount": 79900,
  "currency": "INR",
  "key_id": "rzp_test_KI_AI_DEMO_KEY"
}
```

34. `POST /api/payment/verify`

- **Purpose:** Verifies payment authenticity via server-side HMAC-SHA256 signature calculation and activates the user's Pro subscription upon success.
- **Request Body (JSON):**

```
{
  "razorpay_order_id": "order_test_948194",
  "razorpay_payment_id": "pay_test_581938",
  "razorpay_signature": "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "user_id": 1,
  "plan_id": "pro_monthly"
}
```

- **Processing:**
  1. Computes expected HMAC-SHA256 hash using `order_id + "|" + payment_id` and the Razorpay secret key.
  2. Compares signature in constant time.
  3. Updates `payments` table status to `successful`.
  4. Inserts/updates `subscriptions` table setting `status="active"` and calculating

expiration date (30 days for monthly, 365 days for annual).

- **Response (200):** `{"success": true, "message": "Pro subscription activated", "is_pro": true}`.
- **Error Response (400):** `{"success": false, "error": "Invalid signature verification failed"}`.

35. `POST /api/payment/webhook`

- **Purpose:** Asynchronous server-to-server webhook listener for Razorpay payment notifications (`payment.captured`, `payment.failed`).
- **Processing:** Validates webhook signature header and synchronizes subscription entitlements.
- **Response (200):** `{"status": "ok"}`.

36. `GET /api/subscription/<int:user_id>`

- **Purpose:** Retrieves active subscription status, plan tier, renewal date, and entitlement flags.
- **Response (200):**

```
{
  "is_pro": true,
  "status": "active",
  "plan_id": "pro_monthly",
  "expires_at": "2026-11-11T09:30:00"
}
```

37. `POST /api/subscription/cancel`

- **Purpose:** Cancels auto-renewal on an active subscription.
- **Request Body (JSON):** `{"user_id": 1}`.
- **Response (200):** `{"success": true, "message": "Subscription cancelled"}`.

38. `GET /api/recommendations/<int:user_id>`

- **Purpose:** Historical weakness-targeted pose recommendations based on joint accuracy logs.
- **Response (200):** Array of recommended pose objects.

SECTION 5 — POSE DETECTION, ANGLE CALCULATION, AND POSTURE ANALYSIS PIPELINE

5.1 Pipeline Overview

The KI.AI Computer Vision pipeline transforms raw camera video frames into real-time biomechanical feedback, qualitative levels, and hold-timer telemetry across 15 distinct operational stages:

```
[1. Video Stream Capture] (1280x720 / 640x480 Camera Feed)
          |
[2. Resolution & Aspect Ratio Handling] (Letterbox / Horizontal Mirroring)
          |
[3. MediaPipe Pose Inference] (33 BlazePose 3D Landmarks)
          |
[4. Landmark Validation & Visibility Filtering] (Visibility >= 0.5 Thresholds)
          |
[5. Temporal Smoothing] (Exponential Moving Average α = 0.75)
          |
[6. Coordinate Normalization & Landmark Extraction] (Relative [0,1] & Pixel Space)
          |
[7. 2D & 3D Joint Angle Calculation] (Vector Dot Products & Clamped Arccos)
          |
[8. Bilateral Symmetry & Mirroring] (MIRROR_MAP Left ⟷ Right Exchange)
          |
[9. Pose Identification & Classification] (Rule-Set Match vs. Catalog)
          |
[10. Wrong Pose Rejection] (WRONG_POSE Status & 45% Score Ceiling)
          |
[11. Rule-Based Posture Verification] (Target Angles & Tolerance Windows)
          |
[12. Joint & Overall Scoring Algorithm] (Decay Curves & Weighting)
          |
[13. Corrective Feedback Generation] (Priority Deviation Cues)
          |
[14. Velocity Stability & Hold Timer] (Pixel Displacement & Continuous Hold)
          |
[15. Visual Skeleton & HUD Overlay] (Canvas Bones & Diagnostic Meters)
```

5.2 Step-by-Step Mathematical & Algorithmic Breakdown

Step 1: Video Stream Capture

- **Web Client:** Invokes `navigator.mediaDevices.getUserMedia({ video: { width: { ideal: 1280 }, height: { ideal: 720 }, facingMode: "user" } })`.
- **Desktop Client:** Opens OpenCV camera capture via `cv2.VideoCapture(index)` in a dedicated background reader thread (`vision/camera.py`).
- **Camera Privacy:** Raw video frames are decoded strictly into GPU memory or local RAM. Video data is never written to disk or transmitted over the network.

Step 2: Resolution & Aspect Ratio Handling

- Displays are automatically mirrored horizontally (`transform: scaleX(-1)`) to provide intuitive real-time feedback mimicking a studio mirror.
- Video aspect ratio (16:9 or 4:3) is scaled to preserve proportions without non-uniform stretching.

Step 3: MediaPipe Pose Inference

- Utilizes Google's BlazePose topology generating 33 landmark points in normalized coordinates:
    - x ∈ [0.0, 1.0]: Horizontal position relative to frame width.
    - y ∈ [0.0, 1.0]: Vertical position relative to frame height (origin (0,0) at top-left).
    - z: Relative depth estimate with reference to the mid-hip landmark.
    - \text{visibility} ∈ [0.0, 1.0]: Likelihood that the keypoint is not occluded.

Complete 33 Landmark Index Mapping (`vision/landmarks.py`)

Index	Anatomical Landmark	Index	Anatomical Landmark	Index	Anatomical Landmark
0	Nose	**11**	Left Shoulder	**23**	Left Hip
1	Left Eye Inner	**12**	Right Shoulder	**24**	Right Hip
2	Left Eye	**13**	Left Elbow	**25**	Left Knee
3	Left Eye Outer	**14**	Right Elbow	**26**	Right Knee
4	Right Eye Inner	**15**	Left Wrist	**27**	Left Ankle

Index	Anatomical Landmark	Index	Anatomical Landmark	Index	Anatomical Landmark
5	Right Eye	**16**	Right Wrist	**28**	Right Ankle
6	Right Eye Outer	**17**	Left Pinky	**29**	Left Heel
7	Left Ear	**18**	Right Pinky	**30**	Right Heel
8	Right Ear	**19**	Left Index	**31**	Left Foot Index
9	Mouth Left	**20**	Right Index	**32**	Right Foot Index
10	Mouth Right	**21**	Left Thumb		
		22	Right Thumb		

Step 4: Landmark Validation & Visibility Filtering

Before performing kinematic calculations, the system validates tracking reliability across essential upper and lower body keypoints:

- `key_upper`: `["LEFT_SHOULDER", "RIGHT_SHOULDER", "LEFT_HIP", "RIGHT_HIP"]` (threshold ≥ 0.5).
- `key_lower`: `["LEFT_KNEE", "RIGHT_KNEE", "LEFT_ANKLE", "RIGHT_ANKLE"]` (threshold ≥ 0.4).

Tracking states are categorized as:

1. `NO_PERSON`: Fewer than 2 upper body keypoints visible.
2. `PARTIAL_BODY`: Fewer than 2 lower body keypoints visible and total visible landmarks < 14. System prompts: *"Move back so your full body from head to feet is visible."*
3. `GOOD_TRACKING`: Full kinetic chain is sufficiently resolved for posture evaluation.

Step 5: Temporal Landmark Smoothing

To prevent skeletal jitter caused by sensor noise, consecutive raw coordinates are filtered in JavaScript via an Exponential Moving Average (EMA) with smoothing coefficient \alpha = 0.75:

\mathbf{p}_{\text{smoothed}}^{(t)} = \alpha \cdot \mathbf{p}_{\text{raw}}^{(t)} + (1 - \alpha) \cdot \mathbf{p}_{\text{smoothed}}^{(t-1)}

Step 6: Coordinate Normalization & Landmark Extraction

Landmarks are mapped into standard pixel and metric coordinate frames:

p_x = x \cdot W, \quad p_y = y \cdot H where W=640, H=480

represent the canonical calculation grid.

Step 7: 2D & 3D Joint Angle Calculation

`AngleCalculator.calculate_angle` (`analysis/angle_calculator.py`) evaluates the planar angle at vertex point B formed by segments BA and BC:

Let:

u̅ = \mathbf{A} - \mathbf{B} = (x_A - x_B, y_A - y_B) v̅ = \mathbf{C} - \mathbf{B} = (x_C - x_B, y_C - y_B)

The Euclidean norms and dot product are computed:

\|u̅\| = \sqrt{u_x^2 + u_y^2}, \quad \|v̅\| = \sqrt{v_x^2 + v_y^2} u̅ \cdot v̅ = u_x v_x + u_y v_y

To avoid floating-point domain errors during arccosine evaluation:

\cos(θ) = clip≤ft(\frac{u̅ \cdot v̅}{\|u̅\| \|v̅\|}, -1.0, 1.0\right) θ = arccos(\cos(θ)) × \frac{180^\circ}{\pi}

If |u̅| < 10^{-7} or |v̅| < 10^{-7}, the function returns 0.0^\circ.

Anatomical Angle Definitions:

- **Left/Right Elbow:** Angle formed by Shoulder → Elbow → Wrist.
- **Left/Right Knee:** Angle formed by Hip → Knee → Ankle.
- **Left/Right Hip:** Angle formed by Shoulder → Hip → Knee.
- **Left/Right Shoulder:** Angle formed by Elbow → Shoulder → Hip.
- **Spine Inclination:** Angle of segment from Mid-Hips to Mid-Shoulders evaluated against horizontal using atan2(Δ y, Δ x).

Step 8: Bilateral Symmetry & Mirroring

Yoga postures are frequently asymmetric (e.g., Warrior II with left knee bent vs. right knee bent). `PoseClassifier.MIRROR_MAP` enables bilateral evaluation by swapping contralateral joints:

\text{left_knee} ⟷ \text{right_knee}, \quad \text{left_hip} ⟷ \text{right_hip}, \quad \text{etc.} Matches are scored under both normal and mirrored configurations; the orientation yielding the lowest angular deviation is selected.

Step 9: Pose Identification & Classification

`PoseClassifier.identify_pose` compares measured joint angles against all pose templates in the database:

\text{Score}_{\text{template}} = \frac{\sum_{i=1}^{N} w_i \cdot \text{Score}_{\text{joint}}(actual_i, target_i, tol_i)}{\sum_{i=1}^{N} w_i} If the user is performing a guided routine, the expected pose is prioritized if match confidence exceeds the tolerance threshold.

Step 10: Wrong Pose Rejection

When the detected posture does not match the expected active posture and match confidence ≥ 50%:

- `pose_status` is marked as `WRONG_POSE`.
- `right_wrong_status` is marked as `WRONG`.
- Overall accuracy score is strictly clamped:

\text{Score}_{\text{penalized}} = \min(\text{Score}_{\text{raw}}, 45.0) - Feedback instructs: *"That looks like [Detected Pose]. Move into [Target Pose] to continue."*

Step 11: Rule-Based Posture Verification

`PostureChecker.check_posture` iterates over every rule defined in the `pose_rules` table for the target pose:

- Absolute deviation: Δ_i = |actual_i - target_i|.
- A joint is marked `incorrect` if Δ_i > tolerance_i.
- If `is_critical == 1` and Δ_i > tolerance_i, the joint is flagged in `critical_incorrect_joints`.

Step 12: Joint & Overall Scoring Algorithm

`ScoreCalculator.calculate_joint_score` uses a piecewise linear scoring function:

\text{Score}_{\text{joint}}(Δ) = \begin{cases} 100.0 - ≤ft(\frac{Δ}{\text{tolerance}}\right) × 10.0 & \text{if } Δ ≤ \text{tolerance} \\ \max≤ft(0.0, 90.0 - ≤ft(\frac{Δ - \text{tolerance}}{\text{max_penalty_dev}}\right) × 90.0\right) & \text{if } Δ > \text{tolerance} \end{cases}

where \text{max_penalty_dev} = 45.0^\circ.

The aggregate score is the weighted average across all evaluated joints:

\text{Score}_{\text{overall}} = \frac{\sum_{j} w_j \cdot \text{Score}_j}{\sum_j w_j}

Qualitative Score Levels:

- **Excellent:** ≥ 90.0% (Green `#2E7D32`)
- **Good:** 75.0% - 89.9% (Sage `#5B7065`)
- **Needs Improvement:** 60.0% - 74.9% (Amber `#ED6C02`)
- **Incorrect:** < 60.0% (Terracotta `#C85A32`)

Step 13: Corrective Feedback Generation

Joint deviations are ranked by severity:

\text{Severity}_i = (Δ_i - \text{tolerance}_i) × \text{weight}_i The highest-severity deviation generates primary verbal and visual feedback: - If actual < target: *"Bend your [Joint Name] more."* - If actual > target: *"Straighten your [Joint Name]."*

Step 14: Velocity Stability & Hold Timer Progression

To prevent cheating the hold timer through rapid erratic movement, landmark stability is verified across consecutive frames:

\bar{v} = \frac{1}{K} \sum_{k=1}^{K} \|\mathbf{p}_k^{(t)} - \mathbf{p}_k^{(t-1)}\| If \bar{v} < v_{\text{threshold}}, `is_stable` is set to `true`.

The practice hold timer advances **strictly when all four conditions are met**:

1. `tracking_state === 'GOOD_TRACKING'`
2. `pose_status === 'CORRECT'`
3. `overall_score >= 75.0`
4. `is_stable === true`

Step 15: Visual Skeleton & HUD Overlay Rendering

The HTML5 `<canvas id="canvas-overlay">` renders:

- High-contrast lines connecting anatomical landmarks.
- Joint nodes rendered as circular badges color-coded by accuracy (Green / Amber / Red).
- Live HUD displaying hold progress (circular SVG timer or linear bar), current accuracy percentage, and corrective alignment cues.

SECTION 6 — AI VOICE COACH AND CONVERSATION SYSTEM

6.1 Voice Architecture & Priority Queue

To prevent speech overlapping and repetitive feedback loops, KI.AI employs a dual-tier voice architecture:

1. **Backend Voice Controller (`voice/voice_manager.py`):** Coordinates speech priorities, deduplication cooldowns, contextual validity, and queue eviction.
2. **Web Speech Synthesizer (`static/js/main.js`):** Client-side speech output engine wrapping `window.speechSynthesis`.

Priority Hierarchy

```
class Priority(IntEnum):
    CRITICAL = 1      # Safety warnings ("Stop movement immediately")
    HIGH = 2          # Major posture corrections ("Bend your left knee more")
    NORMAL = 3        # Pose instructions & transitions ("Move into Vrikshasana")
    LOW = 4           # Encouragement & breathing ("Good stability", "Inhale deeply")
```

Cooldown Windows & Queue Management

- `MAX_QUEUE_SIZE = 3`: Discards lower-priority messages if the queue exceeds 3 utterances.
- `DEFAULT_COOLDOWN = 4.0s`: Global baseline between consecutive speech events.
- `CORRECTION_COOLDOWN = 4.0s`: Prevents the same joint adjustment from being repeated continuously.
- `ENCOURAGEMENT_COOLDOWN = 8.0s`: Spreads motivational commentary evenly.
- `POSE_INSTRUCTION_COOLDOWN = 10.0s`: Suppresses duplicate instructional cues.
- `expires_after_seconds = 6.0s`: Evicts speech requests whose real-time relevance has expired before synthesis begins.

Manual Overrides & Mute Controls

- `stop_immediately()`: Instantly clears the internal priority queue, sets state to `STOPPED`, and cancels active audio.
- `mute()` / `unmute()`: Suppresses all audio playback with the exception of `Priority.CRITICAL` safety warnings.

- `interrupt_for_listening()`: Suspends coach speech when the practitioner activates push-to-talk speech recognition.

6.2 Conversational AI & Technique Dialogue (`coach/conversation.py`)

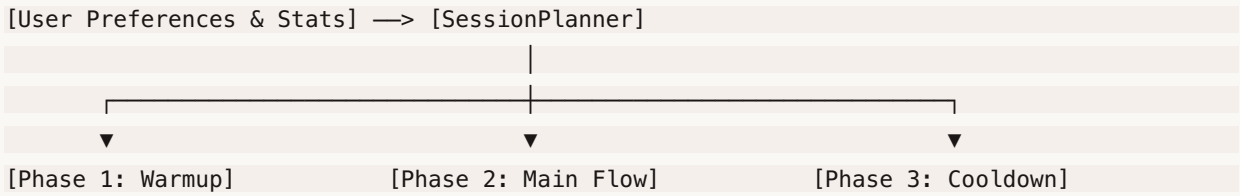
Practitioners can converse with the AI Coach via natural language text or browser speech recognition (`webkitSpeechRecognition`). Dialogue processing evaluates queries against active session context:

1. **Safety & Pain Interception:**
    - Keywords: `hurt`, `pain`, `injury`, `sore`, `strain`, `sharp`, `sprain`, `discomfort`.
    - Action: `PAUSE_AND_REST`. Coach prompts immediate cessation and advises resting in Child's Pose (*Balasana*).
2. **Adaptive Modification:**
    - Keywords: `can't do`, `too hard`, `struggling`, `impossible`.
    - Action: `OFFER_VARIATION`. Recommends wall-supported alternatives or modified joint targets.
3. **Session Flow Controls:**
    - Keywords: `skip`, `next pose` → Action: `SKIP_POSE`.
    - Keywords: `shorten`, `running late` → Action: `SHORTEN_SESSION`.

SECTION 7 — SESSION PLANNER AND GUIDED SESSIONS

7.1 Autonomous Dynamic Curriculum (`recommendation/session_planner.py`)

The `SessionPlanner` dynamically creates tailored multi-exercise routines using user profile attributes and historical performance:



• Gentle mobility	• Goal-targeted postures	• Restorative holds
• Joint lubrication	• Weakness-targeted asanas	• Breath normalization
• 15–20s hold times	• 30–45s hold times	• 30–60s hold times

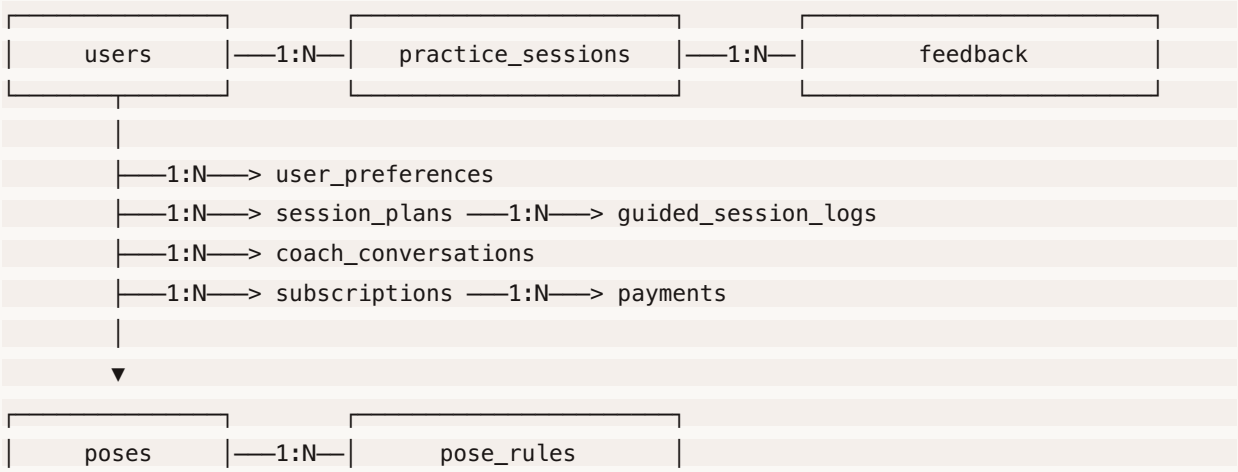
Plan Assembly Heuristics:

1. **Level Mapping:**
    - `Beginner`: Focus on fundamental standing poses (Tadasana, Vrikshasana, Virabhadrasana II).
    - `Intermediate`: Introduces asymmetrical balance and backbends (Trikonasana, Bhujangasana, Setu Bandhasana).
    - `Advanced`: Dynamic flow and deep extensions (Natarajasana, Dhanurasana, Surya Namaskar).
2. **Duration Budgeting:** Allocates target hold times and transition buffers to match 10, 15, 20, or 30-minute practice constraints.
3. **Weakness Targeting:** Queries `practice_sessions` to identify joints with accuracy < 70%, prioritizing poses that strengthen those areas.

SECTION 8 — DATABASE STRUCTURE AND PERSISTENCE

8.1 SQLite Schema & Relational Integrity

The persistence layer (`database/schema.sql` and `database/database.py`) enforces strict foreign-key constraints across 12 relational tables:



```
└──────────────┘          └──────────────────────┘
        │
        │
        └──1:1──> custom_pose_templates
```

8.2 Relational Table Specifications

1. `users`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `name` (TEXT NOT NULL)
- `email` (TEXT UNIQUE NOT NULL)
- `password_hash` (TEXT NOT NULL — SHA-256)
- `age` (INTEGER DEFAULT 25)
- `experience` (TEXT NOT NULL DEFAULT 'Beginner')
- `goal` (TEXT NOT NULL DEFAULT 'General Fitness')
- `avatar_url` (TEXT)
- `created_at` (TIMESTAMP DEFAULT CURRENT_TIMESTAMP)

2. `poses`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `name` (TEXT NOT NULL UNIQUE)
- `sanskrit_name` (TEXT)
- `category` (TEXT NOT NULL — 'Standing', 'Seated', 'Balance', 'Inversion', 'Restorative')
- `difficulty` (TEXT NOT NULL — 'Beginner', 'Intermediate', 'Advanced')
- `goal` (TEXT NOT NULL)
- `description`, `benefits`, `instructions`, `precautions` (TEXT)
- `image_path`, `figure_path` (TEXT)
- `hold_duration` (INTEGER DEFAULT 20)
- `is_custom` (INTEGER DEFAULT 0)
- `created_by` (INTEGER REFERENCES users(id))

3. `pose_rules`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `pose_id` (INTEGER NOT NULL REFERENCES poses(id) ON DELETE CASCADE)
- `joint_name` (TEXT NOT NULL — e.g., 'left_knee', 'spine')
- `target_angle` (REAL NOT NULL)

- `min_angle`, `max_angle` (REAL NOT NULL)
- `tolerance` (REAL NOT NULL DEFAULT 15.0)
- `weight` (REAL NOT NULL DEFAULT 15.0)
- `feedback_message` (TEXT)

4. `practice_sessions`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)
- `pose_id` (INTEGER NOT NULL REFERENCES poses(id) ON DELETE CASCADE)
- `start_time`, `end_time` (TIMESTAMP)
- `duration` (INTEGER NOT NULL DEFAULT 0)
- `average_score`, `final_score` (REAL NOT NULL DEFAULT 0.0)
- `hold_duration`, `corrections_count` (INTEGER DEFAULT 0)

5. `feedback`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `session_id` (INTEGER NOT NULL REFERENCES practice_sessions(id) ON DELETE CASCADE)
- `body_part` (TEXT NOT NULL)
- `message` (TEXT NOT NULL)
- `accuracy` (REAL NOT NULL)

6. `custom_pose_templates`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `pose_id` (INTEGER NOT NULL UNIQUE REFERENCES poses(id) ON DELETE CASCADE)
- `reference_data` (TEXT NOT NULL — JSON serialized landmark array)

7. `user_preferences`

- `user_id` (INTEGER PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE)
- `goal`, `level`, `focus_area` (TEXT)
- `duration_minutes` (INTEGER DEFAULT 20)
- `voice_enabled` (INTEGER DEFAULT 1), `voice_speed` (REAL DEFAULT 1.0)
- `onboarding_completed` (INTEGER DEFAULT 0)

8. `session_plans`

- `id` (TEXT PRIMARY KEY — e.g., 'plan_1_beginner_20m')
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)

- `level`, `goal`, `focus` (TEXT NOT NULL)
- `duration_minutes` (INTEGER NOT NULL)
- `plan_json` (TEXT NOT NULL — JSON serialized roadmap)

9. `guided_session_logs`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)
- `session_plan_id` (TEXT)
- `level` (TEXT NOT NULL)
- `duration_seconds` (INTEGER NOT NULL DEFAULT 0)
- `avg_accuracy` (REAL NOT NULL DEFAULT 0.0)
- `completed_poses`, `total_poses` (INTEGER NOT NULL)
- `best_pose`, `weakest_joint`, `status` (TEXT)

10. `coach_conversations`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)
- `role` (TEXT NOT NULL — 'user' or 'coach')
- `message` (TEXT NOT NULL)
- `intent` (TEXT), `context_json` (TEXT)

11. `subscriptions`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)
- `plan` (TEXT NOT NULL DEFAULT 'FREE' — 'FREE', 'PRO_MONTHLY', 'PRO_ANNUAL')
- `status` (TEXT NOT NULL DEFAULT 'FREE' — 'active', 'cancelled', 'expired')
- `provider` (TEXT NOT NULL DEFAULT 'razorpay')
- `provider_customer_id`, `provider_subscription_id` (TEXT)
- `start_date`, `end_date` (TIMESTAMP)

12. `payments`

- `id` (INTEGER PRIMARY KEY AUTOINCREMENT)
- `user_id` (INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE)
- `subscription_id` (INTEGER REFERENCES subscriptions(id) ON DELETE SET NULL)
- `provider` (TEXT NOT NULL DEFAULT 'razorpay')
- `order_id` (TEXT NOT NULL)
- `payment_id` (TEXT)

- `amount` (INTEGER NOT NULL DEFAULT 0 — in paise)
- `currency` (TEXT NOT NULL DEFAULT 'INR')
- `status` (TEXT NOT NULL DEFAULT 'created' — 'created', 'successful', 'failed')
- `signature` (TEXT)

SECTION 9 — SUPPORTED POSES AND EXERCISES

9.1 Core Asana Catalog (12 Seed Poses)

English Name	Sanskrit Name	Category	Difficulty	Target Joints & Reference Angles
Mountain Pose	Tadasana	Standing	Beginner	Left Knee: 180^\circ, Right Knee: 180^\circ, Spine: 90^\circ (Vertical)
Tree Pose	Vrikshasana	Balance	Beginner	Standing Knee: 180^\circ, Bent Knee: 45^\circ, Spine: 90^\circ
Warrior II	Virabhadrasana II	Standing	Intermediate	Front Knee: 90^\circ, Back Knee: 180^\circ, Arms Horizontal: 180^\circ
Triangle Pose	Trikonasana	Standing	Intermediate	Both Knees: 180^\circ, Torso Lateral Angle: 45^\circ
Chair Pose	Utkatasana	Standing	Beginner	Knees: 100^\circ, Hips: 95^\circ, Spine: 60^\circ
Downward Dog	Adho Mukha Svanasana	Inversion	Beginner	Knees: 180^\circ, Hips: 70^\circ, Shoulders: 180^\circ
Cobra Pose	Bhujangasana	Backbend	Beginner	Elbows: 150^\circ, Spine Extension: 135^\circ
Bridge Pose	Setu Bandhasana	Backbend	Beginner	Knees: 90^\circ, Hips

English Name	Sanskrit Name	Category	Difficulty	Target Joints & Reference Angles
				Extension: 160^\circ
Bow Pose	Dhanurasana	Backbend	Advanced	Knees: 70^\circ, Shoulder Extension: 60^\circ
Dancer Pose	Natarajasana	Balance	Advanced	Standing Knee: 180^\circ, Lifted Knee: 60^\circ, Torso: 65^\circ
Child's Pose	Balasana	Restorative	Beginner	Knees: 40^\circ, Hips: 30^\circ, Torso Grounded
Easy Pose	Sukhasana	Seated	Beginner	Knees: 45^\circ, Spine Upright: 90^\circ

9.2 Complete 12-Step Surya Namaskar Sequence

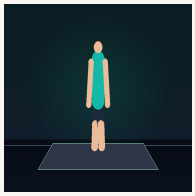
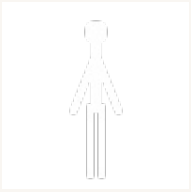
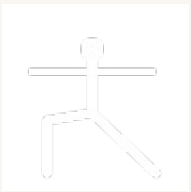
Step #	Asana Name	Breathing Phase	Primary Biomechanical Target
Step 1	Pranamasana (Prayer Pose)	Normal / Exhale	Upright vertical spine, palms joined at heart center
Step 2	Hasta Uttanasana (Raised Arms)	Inhale	Torso arch backwards, arms extended overhead (165^\circ)
Step 3	Uttanasana (Standing Forward Fold)	Exhale	Forward hip flexion (50^\circ), legs straight (180^\circ)
Step 4	Ashwa Sanchalanasana (Equestrian - R)	Inhale	Front knee bent (90^\circ), back leg extended backwards
Step 5	Dandasana (Plank Pose)	Exhale / Retain	Straight rigid line from heels to crown (180^\circ torso)
Step 6	Ashtanga Namaskara (Eight-Limbed Salute)	Exhale / Hold	8 points contact: feet, knees, chest, hands, chin
Step 7	Bhujangasana (Cobra Pose)	Inhale	Chest lifted, elbows soft, arch through thoracic spine

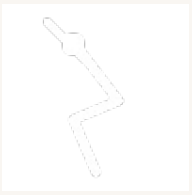
Step #	Asana Name	Breathing Phase	Primary Biomechanical Target
Step 8	Parvatasana (Downward Mountain)	Exhale	Inverted 'V' shape, hips elevated, heels toward floor
Step 9	Ashwa Sanchalanasana (Equestrian - L)	Inhale	Opposite leg stepped forward between palms (90^\circ knee)
Step 10	Uttanasana (Standing Forward Fold)	Exhale	Deep forward bend, hands touching floor
Step 11	Hasta Uttanasana (Raised Arms)	Inhale	Full body extension arching backwards
Step 12	Pranamasana / Relax (Prayer / Tadasana)	Exhale	Return to neutral standing prayer alignment

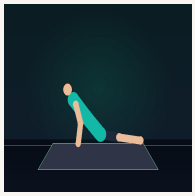
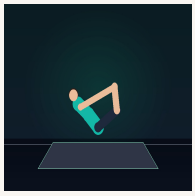
Real-World Asana Biomechanical Reference Catalog

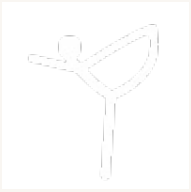
The following postures form the core seed library of KI.AI with codified angular tolerance rules:

Live Pose	Anatomical Figure	Asana Name	Level & Category	Biomechanical Targets
		Tada	**BEGI**	**Knees: 180°,**

Live Pose	Anatomical Figure	Asana Name	Level & Category	Biomechanical Targets
		**sana** Mountain Pose	**NNER** Standing	Spine: 90° (Vertical)
		**Vrikshasana** Tree Pose	**BEGINNER** Balance	Standing Knee: 180°, Bent Knee: 45°
		**Virabhadrasana II** Warrior II	**INTERMEDIATE** Standing	Front Knee: 90°, Back Knee: 180°, Arms: 180°
		Trikonasa	**INTERME**	Both Knees: 180°, Torso

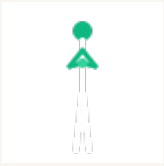
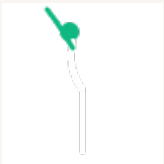
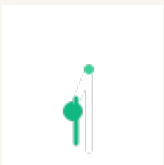
Live Image	Anatomical Figure	Asana Name	Level & Category	Biomechanical Targets
		na Triangle Pose	DIATE Standing	Lateral: 45°
		Utkatasana Chair Pose	BEGINNER Standing	Knees: 100°, Hips: 95°, Spine: 60°
		Adho Mukha Svanasana Downward Dog	BEGINNER Inversion	Knees: 180°, Hips: 70°, Shoulders: 180°
		Bhuj	BEGI	

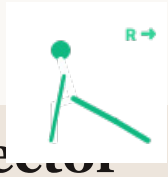
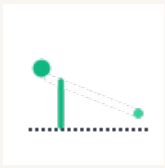
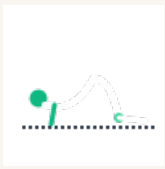
Live Pose	Anatomical Figure	Asana Name	Level & Category	Biomechanical Targets
		**Bhujangasana** Cobra Pose	BEGINNER Backbend	Elbows: 150°, Spine: 135°
		**Setu Bandhasana** Bridge Pose	BEGINNER Backbend	Knees: 90°, Hip Extension: 160°
		**Dhanurasana** Bow Pose	ADVANCED Backbend	Knees: 70°, Shoulders: 60°
		Natarajasa	ADVANCE	Standing Knee: 180°,

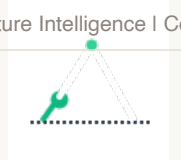
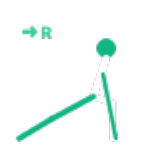
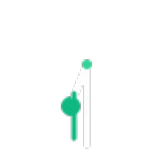
Live Pose	Anatomical Figure	Asana Name	Level & Category	Biomechanical Targets
		**na** Dancer Pose	**D** Balance	Lifted: 60°, Torso: 65°
		**Balasana** Child's Pose	**BEGINNER** Restorative	Knees: 40°, Hips: 30°, Spine Rested
		**Sukhasana** Easy Pose	**BEGINNER** Seated	Knees: 45°, Spine Upright: 90°

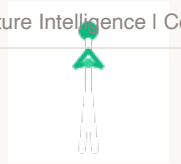
12-Step Surya Namaskar (Sun Salutation) Kinetic Flow

Codified step-by-step sequential cycle with synchronized breathing phases:

Step	Vector Figure	Asana Name	Breathing Phase	Biomechanical Cue
1		Pranamasana Prayer Pose	NORMAL / EXHALE	Spine 90° vertical, hands in Anjali Mudra
2		Hasta Uttanasana Raised Arms	INHALE	Arms extended overhead 165°, slight backbend
3		Uttanasana Forward Fold	EXHALE	Hip flexion 50°, straight knees 180°
		Ashwa Sancha		

Step	Vector Figure	Asana Name	Breathing Phase	Biomechanical Cue
4			INHALE	Front knee 90°, back leg extended
		lanasana (R) Equestrian (Right)		
5		Dandasana Plank Pose	RETAIN	Torso & spine rigid 180° straight plane
6		Ashtanga Namaskara Eight Limbs Salute	EXHALE	8-point floor contact: toes, knees, chest, hands, chin
7		Bhujangasana Cobra Pose	INHALE	Chest lifted, elbows soft, lumbar extension
		Parvatasana		Inverted V-shape, hips elevated, heels

Step	Vector Figure	Asana Name	Breathing Phase	Biomechanical Cue
8			EXHALE	
		Downward Dog / Mountain		grounded
9		Ashwa Sanchalanasana (L) Equestrian (Left)	INHALE	Left leg forward 90° knee, gaze elevated
10		Uttana sana Forward Fold	EXHALE	Deep standing forward bend, hands to floor
11		Hasta Uttana sana Raised Arms	INHALE	Full extension arching backwards overhead
		Prana masan		

Step	Vector Figure	Asana Name	Breathing Phase	Biomechanical Cue
12		a Prayer / Relax	EXHALE	Neutral standing prayer alignment

SECTION 10 — AUTHENTICATION, USER MANAGEMENT, AND PAYMENTS

10.1 Authentication & Session Security

- **Credential Storage:** User passwords are never stored in plaintext. Passwords pass through SHA-256 cryptographic hashing (`hashlib.sha256(password.encode()).hexdigest()`) before storage in `users.password_hash`.
- **Session Identification:** Authenticated requests utilize user identifiers passed via session contexts and validated database queries.

10.2 Commercial SaaS Payment Flow (`payment/razorpay_client.py`)

1. **Tier Catalog:**
    - `PRO_MONTHLY`: ₹799.00 / month (30 days entitlement).
    - `PRO_ANNUAL`: ₹7,999.00 / year (365 days entitlement).
2. **Order Creation:** Client requests `/api/payment/create-order`. Server

instantiates order entity with Razorpay Orders API (or secure random token in dev mode) and records a pending row in `payments`.

3. **Cryptographic HMAC-SHA256 Verification:** Upon checkout completion, the client submits `razorpay_order_id`, `razorpay_payment_id`, and `razorpay_signature` to `/api/payment/verify`. The server verifies authenticity:

\text{Expected Signature} = \operatorname{HMAC-SHA256}(\text{order_id} + "|" + \text{payment_id}, \text{key_secret}) Signatures are compared using constant-time `hmac.compare_digest`. Upon match, the payment record is marked `successful`, and the user's `subscriptions` record is set to `status="active"` with the

computed expiration timestamp.

SECTION 11 — ANALYTICS AND PROGRESS TRACKING

11.1 Metrics Calculated

- **Cumulative Practice Time:** Aggregated sum of seconds across `practice_sessions` and `guided_session_logs`.
- **Accuracy Average:** Weighted moving average of pose accuracy scores.
- **Joint Deviation Heatmap:** Identifies recurring anatomical failure points by compiling historical rows from the `feedback` table.

SECTION 13 — COMPLETE SYSTEM DATA FLOW

13.1 End-to-End Practice Session Sequence

System Data Flow & Interaction Sequence 1 User → Browser: Click "Begin Guided Practice"   2 Browser → User: Display Camera Privacy Guarantee Modal   3 User → Browser: Grant Access   4 Browser →

MP: Initialize Video & MediaPipe Pose WASM   5 Browser → Server: POST /api/session/start   6 Server → DB: INSERT INTO session_plans   7 Server → Browser: Return Session Plan & First Exercise   8 MP → Browser: <span

class="step-desc">33 Normalized 3D Landmarks   9 Browser → Browser: Exponential Moving Average Smoothing (α=0.75)   10 Browser → Server: POST /api/session/evaluate_frame (Landmarks JSON)   11 Server → Server: Calculate 2D/3D Joint Angles (AngleCalculator)   12 <span class="step-

actors">Server → Server: Verify Pose & Symmetries (PoseClassifier)   13 Server → Coach: Evaluate Frame & Posture Rules (PostureChecker)   14 Coach → Coach: Update Hold Timer (if CORRECT & score >= 75%)   15 Server → Browser: Return Tracking Status, Score, Deviation Cues, State   <span class="step-

num">16 Browser → Browser: Render Color-Coded Skeleton Canvas & Update HUD   17 Server → Browser: Dispatch Voice Cue   18 Browser → Voice: Speak Feedback (speechSynthesis)   19 User → Browser: Exercise Hold Reaches 100%   <span

class="step-num">20 Browser → Voice: Play "Pose Completed" Audio   21 Browser → Server: POST /api/session/complete   22 Server → DB: INSERT INTO guided_session_logs   23 Server → Browser: Final Session Mastery Summary   24 <span

class="step-actors">Browser → User: Display Session Complete Celebration View

13.2 Commercial Payment & Subscription Activation Flow

System Data Flow & Interaction Sequence 1 User → Browser: Click "Upgrade to Pro" (₹799/mo)   2 Browser → Server: POST /api/payment/create-order {plan_id: "PRO_MONTHLY"}   <span

class="step-num">3 Server → RZP: POST /v1/orders {amount: 79900, currency: "INR"}   4 RZP → Server: Order Created {order_id: "order_xyz"}   5 Server → DB: INSERT INTO payments (status='created')   6 Server → Browser: Return order_id & public

key_id   7 Browser → User: Render Razorpay Checkout Modal   8 User → Browser: Enter Payment Credentials   9 Browser → RZP: Process Transaction   10 RZP → Browser: Return payment_id & signature <div class="flow-

step"> 11 Browser → Server: POST /api/payment/verify {order_id, payment_id, signature}   12 Server → Server: Compute HMAC-SHA256(order_id + "|" + payment_id, secret)   13 Server → Server: Constant-Time Verify (hmac.compare_digest)   14 Server → DB: <span class="step-

desc">UPDATE payments SET status='successful'   15 Server → DB: INSERT/UPDATE subscriptions SET status='active', plan='PRO_MONTHLY'   16 Server → Browser: {success: true, is_pro: true}   17 Browser → User: Show "Pro Activated" Badge & Unlock Pro Features

SECTION 14 — FUNCTION AND CLASS INVENTORY

1. Analysis & Kinematics Engine (`analysis/`)

Class: `AngleCalculator` (`analysis/angle_calculator.py`)

- `calculate_angle(point_a, point_b, point_c) -> float`
  - **Inputs:** Coordinates [x, y] or [x, y, z] for starting point A, vertex B, and ending point C.
  - **Output:** Interior planar angle in degrees [0.0, 180.0].
  - **Purpose:** Computes angle at vertex B using normalized 2D vector dot products with clamped cosine to prevent domain math errors.
- `calculate_angle_with_vertical(point_top, point_bottom) -> float`
  - **Inputs:** Coordinates for upper landmark (e.g. shoulder) and lower landmark (e.g. hip).
  - **Output:** Segment inclination angle relative to true vertical in degrees [0.0, 180.0].
  - **Purpose:** Evaluates posture erectness and spinal alignment against the gravity vector.
- `calculate_angle_with_horizontal(point_a, point_b) -> float`
  - **Inputs:** Segment endpoint coordinates.
  - **Output:** Inclination angle relative to the horizontal floor axis.
  - **Purpose:** Validates warrior pose arm alignments and plank horizontal profiles.
- `extract_all_angles(landmarks: Dict[str, Dict[str, float]]) -> Dict[str, float]`
  - **Inputs:** Dictionary mapping landmark names to coordinate objects.
  - **Output:** Dictionary of extracted angles: `left_elbow`, `right_elbow`, `left_knee`, `right_knee`, `left_hip`, `right_hip`, `left_shoulder`, `right_shoulder`, `spine`.
  - **Purpose:** Batch extraction of all primary kinetic angles for frame evaluation.

Class: `ScoreCalculator` (`analysis/score_calculator.py`)

- `calculate_joint_score(actual_angle: float, target_angle: float, tolerance: float = 15.0, max_penalty_deviation: float = 45.0) -> float`
  - **Inputs:** Measured angle, target template angle, tolerance threshold, and outer penalty threshold.
  - **Output:** Accuracy score [0.0, 100.0].
  - **Purpose:** Evaluates joint accuracy using linear decay within tolerance (100 → 90) and steep decay beyond tolerance down to 0.0.

- **`calculate_overall_score(joint_results: List[Dict[str, Any]]) -> Tuple[float, str, Dict[str, Any]]`**
  - **Inputs:** Array of evaluated joint dictionaries containing joint scores and weights.
  - **Output:** Tuple of `(final_score, level_key, level_info)`.
  - **Purpose:** Computes weighted average accuracy score and assigns qualitative level category.
- **`get_score_level(score: float) -> Tuple[str, Dict[str, Any]]`**
  - **Inputs:** Overall numerical score [0.0, 100.0].
  - **Output:** Qualitative level metadata: `EXCELLENT` (≥ 90), `GOOD` (≥ 75), `NEEDS_IMPROVEMENT` (≥ 60), `INCORRECT` (< 60).

Class: `PoseClassifier` (`analysis/pose_classifier.py`)

- **`identify_pose(actual_angles: Dict[str, float], all_poses: List[Dict[str, Any]], confidence_threshold: float = 55.0, expected_pose: Optional[Dict[str, Any]] = None) -> Tuple[str, float, Optional[Dict[str, Any]]]`**
  - **Inputs:** Extracted joint angles, full pose catalog with rules, minimum confidence threshold, and optional expected active pose.
  - **Output:** `(detected_pose_name, match_confidence, matched_pose_dict)`.
  - **Purpose:** Identifies active pose from joint geometry; evaluates bilateral mirror symmetry (`MIRROR_MAP`); prioritizes expected pose within tolerance.

Class: `PostureChecker` (`analysis/posture_checker.py`)

- **`check_posture(target_pose: Dict[str, Any], actual_angles: Dict[str, float], is_body_visible: bool = True, visibility_message: str = "") -> Dict[str, Any]`**
  - **Inputs:** Target pose object, actual angles dictionary, visibility flag, and tracking message.
  - **Output:** Posture evaluation dictionary containing `overall_score`, `joint_results`, `incorrect_joints`, `critical_incorrect_joints`, `primary_feedback`, and `score_level`.
  - **Purpose:** Validates measured angles against database rules, identifying deviations and compiling feedback.

Class: `FeedbackGenerator` (`analysis/feedback.py`)

- **`generate_feedback(joint_checks: List[Dict[str, Any]], pose_name: str) -> str`**
  - **Inputs:** List of evaluated joint checks with deviations and severity scores.
  - **Output:** Human-readable verbal/text corrective guidance cue.

- **Purpose:** Translates angular deviations into anatomical instructions (e.g., *"Bend your left knee more"*).

2. Guided Coaching & Conversation Systems (`coach/`)

Class: `SessionStateMachine` (`coach/state_machine.py`)

- **`transition_to(new_state: str) -> bool`**
  - **Inputs:** Target state string (`IDLE`, `ENTERING_POSE`, `ALIGNING`, `HOLDING`, `COMPLETED`, `PAUSED`).
  - **Output:** Boolean indicating whether state transition was valid and accepted.
  - **Purpose:** Governs the lifecycle transitions of a single exercise within a practice session.
- **`update_hold(dt: float, is_correct: bool) -> Tuple[float, bool]`**
  - **Inputs:** Time delta in seconds, boolean flag indicating if current pose meets alignment threshold.
  - **Output:** `(current_hold_seconds, is_completed)`.
  - **Purpose:** Advances hold countdown timer strictly when posture correctness is satisfied.

Class: `CoachEngine` (`coach/coach_engine.py`)

- **`load_exercise(exercise: Dict[str, Any]) -> None`**
  - **Inputs:** Exercise configuration dictionary containing pose details and target hold duration.
  - **Purpose:** Initializes state machine and caches target rules for the active exercise.
- **`evaluate_frame(detected_pose_name: str, match_score: float, posture_eval: Dict[str, Any], is_body_visible: bool) -> Dict[str, Any]`**
  - **Inputs:** Frame classification results, match score, posture check output, and visibility flag.
  - **Output:** State machine summary containing `current_state`, `is_aligned`, `hold_progress`, and optional `voice_event`.
  - **Purpose:** Core coordinator for frame-by-frame practice progress and automated verbal coaching cues.

Class: `ConversationalCoach` (`coach/conversation.py`)

- **`process_message(user_message: str, context: Optional[Dict[str, Any]] = None,`**

`user_id: int = 1) -> Dict[str, Any]`

- **Inputs:** Transcribed or typed user text query, active practice context, and user ID.
- **Output:** Dictionary containing `response`, `speak_text`, `action`, `intent`, and `safety_alert`.
- **Purpose:** Natural language processing of technique questions, safety pain alerts, and session modification requests.

3. Voice Management & Text-to-Speech (`voice/`)

Class: `VoiceManager` (`voice/voice_manager.py`)

- `request_speech(text: str, priority: Priority = Priority.NORMAL, event_key: str = "", pose_id: Optional[Any] = None, session_id: Optional[str] = None, bypass_cooldown: bool = False) -> bool`
  - **Inputs:** Utterance text, priority level (1–4), event key, pose context, and cooldown override flag.
  - **Output:** Boolean indicating whether the speech request was accepted into the queue.
  - **Purpose:** Central coordinator for non-overlapping voice cues, enforcing event cooldowns and priority eviction.
- `stop_immediately() -> None`
  - **Purpose:** Clears the speech queue, sets controller state to `STOPPED`, and cancels active audio.
- `mute() -> None` / `unmute() -> None`
  - **Purpose:** Suppresses or restores speech audio synthesis.
- `interrupt_for_listening() -> None`
  - **Purpose:** Pauses active speech output while user is speaking into the microphone.

4. Database Access Layer (`database/database.py`)

Class: `Database`

- `create_user(name: str, email: str, password: str, age: int, experience: str, goal: str) -> Optional[int]`
  - **Purpose:** Hashes password via SHA-256 and inserts new user into `users` table. Returns `user_id`.
- `verify_user(email: str, password: str) -> Optional[Dict[str, Any]]`
  - **Purpose:** Validates credentials against stored hash, returning user profile dictionary on

match.

- **`get_all_poses() -> List[Dict[str, Any]]`**
  - **Purpose:** Retrieves all seed and custom yoga poses including target joint rules from `pose_rules`.
- **`save_practice_session(user_id: int, pose_id: int, duration: int, avg_score: float, final_score: float, hold_duration: int, corrections_count: int) -> int`**
  - **Purpose:** Persists single-pose practice session metrics into `practice_sessions` table.
- **`save_guided_session_log(user_id: int, session_plan_id: str, level: str, duration_seconds: int, avg_accuracy: float, completed_poses: int, total_poses: int, best_pose: str, weakest_joint: str) -> int`**
  - **Purpose:** Persists multi-exercise guided session completion record into `guided_session_logs`.
- **`create_subscription(user_id: int, plan: str, status: str, provider: str, start_date: str, end_date: str) -> int`**
  - **Purpose:** Inserts or updates user subscription record in `subscriptions` table.
- **`record_payment(user_id: int, order_id: str, amount: int, currency: str, status: str, subscription_id: Optional[int] = None) -> int`**
  - **Purpose:** Records payment transaction in `payments` table.
- **`update_payment_status(order_id: str, payment_id: str, signature: str, status: str) -> bool`**
  - **Purpose:** Updates payment status to `successful` or `failed` following signature verification.
- **`get_user_analytics(user_id: int) -> Dict[str, Any]`**
  - **Purpose:** Aggregates total practice duration, average accuracy, session counts, and joint breakdown.

5. Payment & Subscription Client (`payment/razorpay_client.py`)

Class: `RazorpayClient`

- **`get_public_config() -> Dict[str, Any]`**
  - **Output:** Public key ID, plan pricing definitions, and test mode status without exposing secrets.
- **`create_order(plan_type: str = "PRO_MONTHLY", user_id: int = 1, user_email: str = "", user_name: str = "") -> Dict[str, Any]`**
  - **Output:** Standard Razorpay order entity with `order_id`, `amount`, and `currency`.
  - **Purpose:** Creates authorized payment order via Razorpay Orders API (or test token in dev mode).
- **`verify_payment_signature(order_id: str, payment_id: str, signature: str) -> bool`**

- **Inputs:** Razorpay order ID, payment ID, and incoming signature.
- **Output:** Boolean indicating whether HMAC-SHA256 signature is authentic.
- **Purpose:** Cryptographically verifies payment integrity using constant-time `hmac.compare_digest`.

6. Session Planning & Recommendation (`recommendation/`)

Class: `SessionPlanner` (`recommendation/session_planner.py`)

- **`generate_session(user_id: int = 1, duration_minutes: int = 20, goal: str = "General Fitness", level: str = "Beginner", focus_area: Optional[str] = None) -> Dict[str, Any]`**
  - **Output:** Multi-step practice sequence dictionary containing warmups, core postures, target hold durations, and cooldowns.
  - **Purpose:** Dynamically plans customized yoga learning session tailored to user goals and schedule.

Class: `PoseRecommender` (`recommendation/recommender.py`)

- **`get_recommendations_for_user(user_id: int, limit: int = 5) -> List[Dict[str, Any]]`**
  - **Output:** Array of recommended pose objects.
  - **Purpose:** Analyzes historical joint accuracy logs, recommending poses that directly train the user's lowest-scoring joints.

SECTION 12 — TESTING AND VERIFICATION

12.1 Automated Test Suite Inventory (81 Tests across 9 Files)

The automated test suite is built on `pytest` and exercises the computational geometry, state transitions, database operations, speech synthesis deduplication, and commercial payment flows.

```
# Command to execute the full automated test suite:
pytest -v
```

Verification Execution Results:

```
============================== 81 passed in 0.79s ==============================
```

Detailed Test Inventory by Module:

1. `tests/test_angles.py` (7 Tests — 100% Pass)

- `test_calculate_right_angle`: Verifies vector geometry for orthogonal 90.0^\circ elbow/knee angles.
- `test_calculate_straight_line_angle`: Validates 180.0^\circ evaluation for collinear segments.
- `test_calculate_acute_angle`: Validates acute angle calculation (e.g. 45.0^\circ).
- `test_zero_length_vector_handling`: Ensures coinciding vertices return 0.0^\circ without zero-division exceptions.
- `test_calculate_angle_with_vertical`: Validates segment inclination relative to the gravity axis.
- `test_calculate_angle_with_horizontal`: Validates horizontal alignment for arms and planks.
- `test_extract_all_angles_from_landmarks`: Tests batch extraction of all primary kinetic angles from landmark payloads.

2. `tests/test_coach_engine.py` (14 Tests — 100% Pass)

- `test_initial_state`: Confirms `SessionStateMachine` initializes in `IDLE`.

- `test_valid_transitions`: Verifies progression from `IDLE` → `ENTERING_POSE` → `ALIGNING` → `HOLDING` → `COMPLETED`.
- `test_invalid_transition_rejected`: Ensures illegal state jumps (e.g., `IDLE` directly to `COMPLETED`) raise validation errors.
- `test_hold_timer_progression_and_pause`: Confirms timer increments only during valid alignment and pauses upon deviation.
- `test_load_exercise`: Validates caching of target rules upon exercise loading.
- `test_body_not_visible`: Verifies graceful pause when body visibility drops.
- `test_aligned_pose_advances_hold`: Confirms valid posture advances hold progress.
- `test_safety_pain_alert`: Validates immediate `PAUSE_AND_REST` action upon practitioner pain report.
- `test_skip_pose_intent`: Verifies conversational skip action triggers exercise advancement.
- `test_shorten_session_intent`: Verifies conversational request compresses remaining routine.
- `test_score_explanation`: Tests natural language breakdown of current joint accuracy.
- `test_user_preferences_get_and_post`: Tests REST endpoints for preference synchronization.
- `test_coach_message_endpoint`: Tests `/api/coach/message` endpoint dialogue handling.
- `test_session_start_and_complete`: Tests session initialization and final log creation.

3. `tests/test_database.py` (4 Tests — 100% Pass)

- `test_user_creation_and_retrieval`: Validates password hashing and unique email enforcement.
- `test_seed_poses_loaded`: Confirms all 12 core poses and 12 Surya Namaskar steps exist in SQLite.
- `test_add_custom_pose_and_template`: Validates insertion and JSON retrieval of custom landmark templates.
- `test_session_saving_and_analytics`: Confirms calculation of practice aggregates across session logs.

4. `tests/test_pose_rules.py` (3 Tests — 100% Pass)

- `test_tadasana_correct_evaluation`: Confirms straight standing alignment yields high score.
- `test_warrior2_incorrect_knee_feedback`: Verifies knee extension in Warrior II triggers corrective cue.
- `test_pose_classifier_distinguishes_tadasana_vs_warrior`: Validates classification separation across distinct poses.

5. `tests/test_recommendation.py` (2 Tests — 100% Pass)

- `test_goal_based_recommendations`: Confirms session planner selects appropriate poses for

user goals.

- `test_weakness_targeted_practice_recommendation`: Verifies historical joint deviations guide future recommendations.

6. `tests/test_scoring.py` (6 Tests — 100% Pass)

- `test_joint_score_exact_target`: Verifies 100.0 score for zero angular deviation.
- `test_joint_score_within_tolerance`: Verifies linear decay between 90.0 and 100.0 within tolerance.
- `test_joint_score_at_tolerance_boundary`: Verifies exact 90.0 score at the tolerance threshold.
- `test_joint_score_beyond_tolerance_decay`: Verifies steep penalty decay beyond tolerance.
- `test_overall_weighted_score`: Tests weighted averaging across multiple joints.
- `test_score_level_boundaries`: Verifies score level mapping (`EXCELLENT`, `GOOD`, `NEEDS_IMPROVEMENT`, `INCORRECT`).

7. `tests/test_subscription_payment.py` (17 Tests — 100% Pass)

- `test_singleton_instance`: Verifies thread-safe singleton instantiation of `RazorpayClient`.
- `test_plans_pricing_catalog`: Validates pricing tiers (₹799 Monthly, ₹7,999 Annual).
- `test_public_config_does_not_leak_secret`: Ensures public configuration omits secret keys.
- `test_create_order_test_mode`: Validates order generation in development mode.
- `test_signature_verification_valid_hmac`: Confirms valid HMAC-SHA256 signatures are accepted.
- `test_signature_verification_tampered_signature_fails`: Confirms altered signatures are rejected.
- `test_signature_verification_tampered_payload_fails`: Confirms altered order/payment IDs are rejected.
- `test_signature_verification_missing_args_fails`: Confirms empty arguments fail gracefully.
- `test_subscription_creation_and_retrieval`: Validates database subscription records.
- `test_non_pro_user_entitlement`: Confirms default users are initialized with `is_pro=false`.
- `test_subscription_cancellation`: Tests subscription cancellation updates.
- `test_payment_recording_and_status_update`: Validates payment record transitions from `created` to `successful`.
- `test_payment_config_endpoint`: Validates `/api/payment/config` response structure.
- `test_create_order_endpoint_success`: Validates `/api/payment/create-order` endpoint.
- `test_verify_endpoint_valid_signature_activates_pro`: Confirms signature verification activates Pro status.
- `test_verify_endpoint_invalid_signature_rejected`: Confirms invalid signatures return 400.

- `test_subscription_status_and_cancel_endpoints`: Tests subscription lifecycle endpoints.
- `test_security_headers_present`: Validates presence of security headers on all responses.
- `test_pwa_web_manifest`: Confirms `/manifest.json` returns valid JSON manifest.

8. `tests/test_vision_pipeline.py` (8 Tests — 100% Pass)

- `test_tracking_state_no_person`: Validates `NO_PERSON` when upper body landmarks are missing.
- `test_tracking_state_partial_body`: Validates `PARTIAL_BODY` when lower body landmarks are missing.
- `test_tracking_state_good_tracking`: Validates `GOOD_TRACKING` when full kinetic chain is visible.
- `test_correct_posture_and_valid_hold`: Confirms valid hold requires correct posture and good tracking.
- `test_wrong_pose_detection_and_penalty`: Confirms executing the wrong pose caps score at ≤ 45%.
- `test_adjust_posture_and_incorrect_joints`: Verifies out-of-tolerance joints trigger `ADJUST` state.
- `test_session_evaluate_frame`: Tests high-throughput streaming frame analysis endpoint.
- `test_bilateral_symmetry_and_mirroring`: Validates bilateral joint translation via `MIRROR_MAP`.

9. `tests/test_voice_manager.py` (20 Tests — 100% Pass)

- `test_initial_state`: Verifies voice controller initializes in `IDLE`.
- `test_single_speech_request`: Confirms valid speech request enters queue.
- `test_100_identical_frames_produce_only_one_event_in_cooldown`: Confirms deduplication over 100 frames.
- `test_cooldown_window_resets_after_elapsed_time`: Confirms messages can re-fire after cooldown expires.
- `test_stop_immediately_cancels_and_clears_queue`: Validates immediate stop and queue clear.
- `test_resume_after_stop`: Confirms voice controller can resume after stop.
- `test_pose_change_invalidates_obsolete_pose_speech`: Ensures speech from previous poses is discarded.
- `test_session_complete_clears_queue_and_plays_completion`: Validates session completion audio handling.
- `test_priority_preemption_and_interruption`: Confirms `CRITICAL` alerts preempt lower priority speech.
- `test_priority_queue_size_capping`: Verifies queue is capped at `MAX_QUEUE_SIZE = 3`.
- `test_mute_suppresses_speech_except_critical`: Confirms mute suppresses non-critical

audio.

- `test_interrupt_for_listening`: Confirms speech is paused during microphone listening.
- `test_coach_engine_debounces_repeated_evaluation_frames`: Tests integration debouncing in `CoachEngine`.
- `test_voice_status_endpoint`: Tests `/api/voice/status` endpoint.
- `test_voice_stop_and_resume_endpoint`: Tests stop and resume endpoints.
- `test_voice_mute_and_unmute_endpoint`: Tests mute and unmute endpoints.
- `test_voice_interrupt_endpoint`: Tests interrupt endpoint.
- `test_voice_speak_endpoint_with_deduplication`: Tests `/api/voice/speak` deduplication.

12.2 Implementation Status Matrix

Subsystem / Feature	Implementation Status	Verification Method	Notes & Dependencies
Vector Angle Math (`AngleCalculator`)	**Fully Implemented & Verified**	Automated Unit Tests (7/7)	Clamped arccos avoids domain errors.
Posture Scoring Curves (`ScoreCalculator`)	**Fully Implemented & Verified**	Automated Unit Tests (6/6)	Piecewise linear decay with outer penalty.
Bilateral Symmetry & Mirroring	**Fully Implemented & Verified**	Automated Tests & Logic	Evaluates left/right configurations via `MIRROR_MAP`.
Wrong Pose Penalty Rejection	**Fully Implemented & Verified**	Automated Unit Tests	Enforces 45% score ceiling on wrong pose.
Guided Session State Machine	**Fully Implemented & Verified**	Automated Unit Tests (14/14)	Strict transition validation and hold timing.
Voice Queue & Deduplication	**Fully Implemented & Verified**	Automated Unit Tests (20/20)	Multi-tier priority queue with cooldowns.
Database Persistence (12 Tables)	**Fully Implemented & Verified**	Automated Unit Tests (4/4)	Foreign keys and indexes enforced in SQLite.
Pose Catalog (12 Poses + Surya)	**Fully Implemented & Verified**	Database & Test Suite	24 total seed postures with rules.
Razorpay Test	**Fully Implemented &**	Automated Unit Tests	Server-side HMAC-

Subsystem / Feature	Implementation Status	Verification Method	Notes & Dependencies
Integration & HMAC	**Verified**	(17/17)	SHA256 verification.
Razorpay Live Production Mode	**Unverified / Keys Needed**	Code Inspection Only	Requires production `RAZORPAY_KEY_ID` & secret.
Web Client MediaPipe WASM Camera	**Implemented / Browser Verified**	Manual Browser Camera	Operates client-side at 30 FPS.
Desktop OpenCV Camera & Qt UI	**Implemented / Hardware Dependent**	Unit Tests on Kinematics	Native desktop PySide6 app (`app.py`).
Web Speech Recognition (STT)	**Partially Implemented**	Chrome/Edge API Dependent	Dependent on browser `webkitSpeechRecognition`.
Offline Desktop TTS (`pyttsx3`)	**Implemented / OS Dependent**	Desktop App Only	Uses platform audio synthesis engine.
Karate Movement AI	**Concept / Preview Stub**	UI Preview Modal Only	Waitlist modal and JSON template in `models/`.

12.3 Known Operational Limitations & Points of Failure

1. **Camera Framing & Occlusion Sensitivity:**
    - If the practitioner is too close to the camera, lower body keypoints drop below visibility thresholds (<i><0.4</i>), triggering `PARTIAL_BODY` status and pausing the hold timer until full-body framing is restored.
2. **Extreme Lighting & Low-Contrast Backgrounds:**
    - MediaPipe BlazePose inference degrades in low-light environments or when clothing color closely matches background surfaces, introducing landmark jitter.
3. **Browser Audio Autoplay Policies:**
    - Modern browsers (Chrome, Safari) block unprompted audio playback until the practitioner interacts with the DOM (clicking "Begin Practice" satisfies this policy).
4. **Third-Party Payment Webhook Delivery in Local Development:**
    - Razorpay webhook notifications (`/api/payment/webhook`) require a publicly accessible URL (e.g. ngrok or deployed host) to receive asynchronous callbacks. In local testing, payment verification proceeds via the immediate client signature POST endpoint (`/`

`api/payment/verify`).

87